

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

17

Two-Phase Locking



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

LAST CLASS

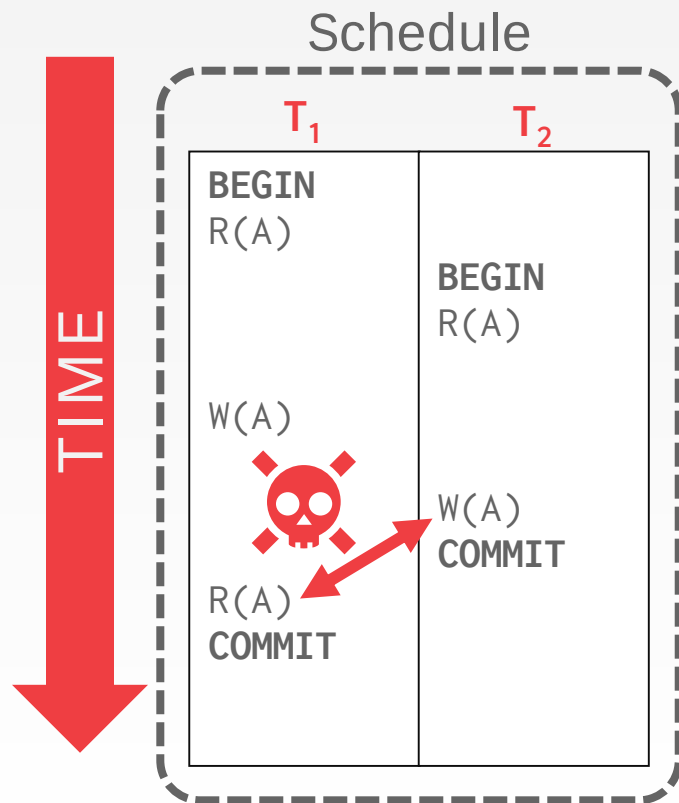
Conflict Serializable

- Verify using either the "swapping" method or dependency graphs.
- Any DBMS that says that they support "serializable" isolation does this.

View Serializable

- No efficient way to verify.
- Andy doesn't know of any DBMS that supports this.

EXAMPLE

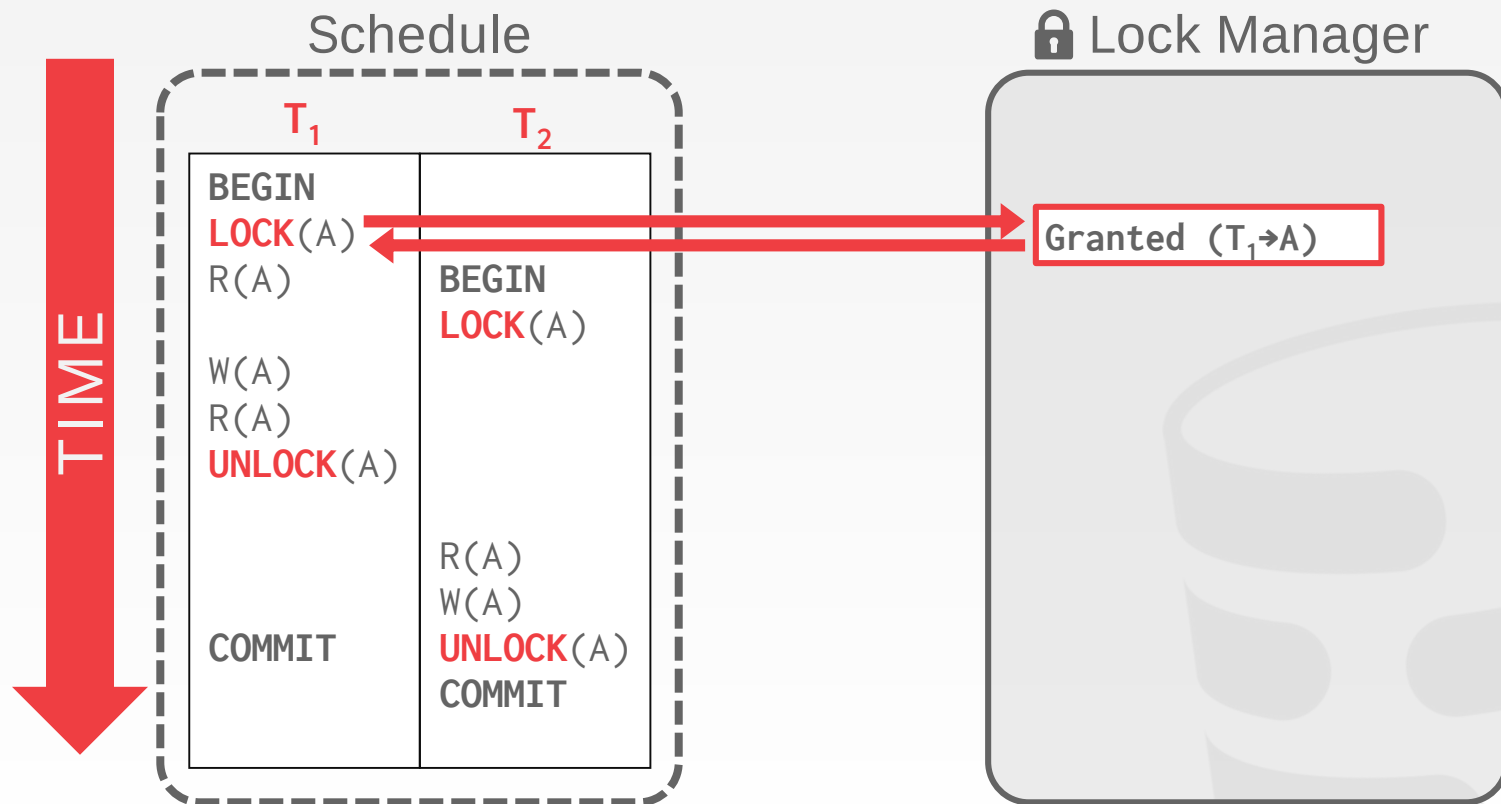


OBSERVATION

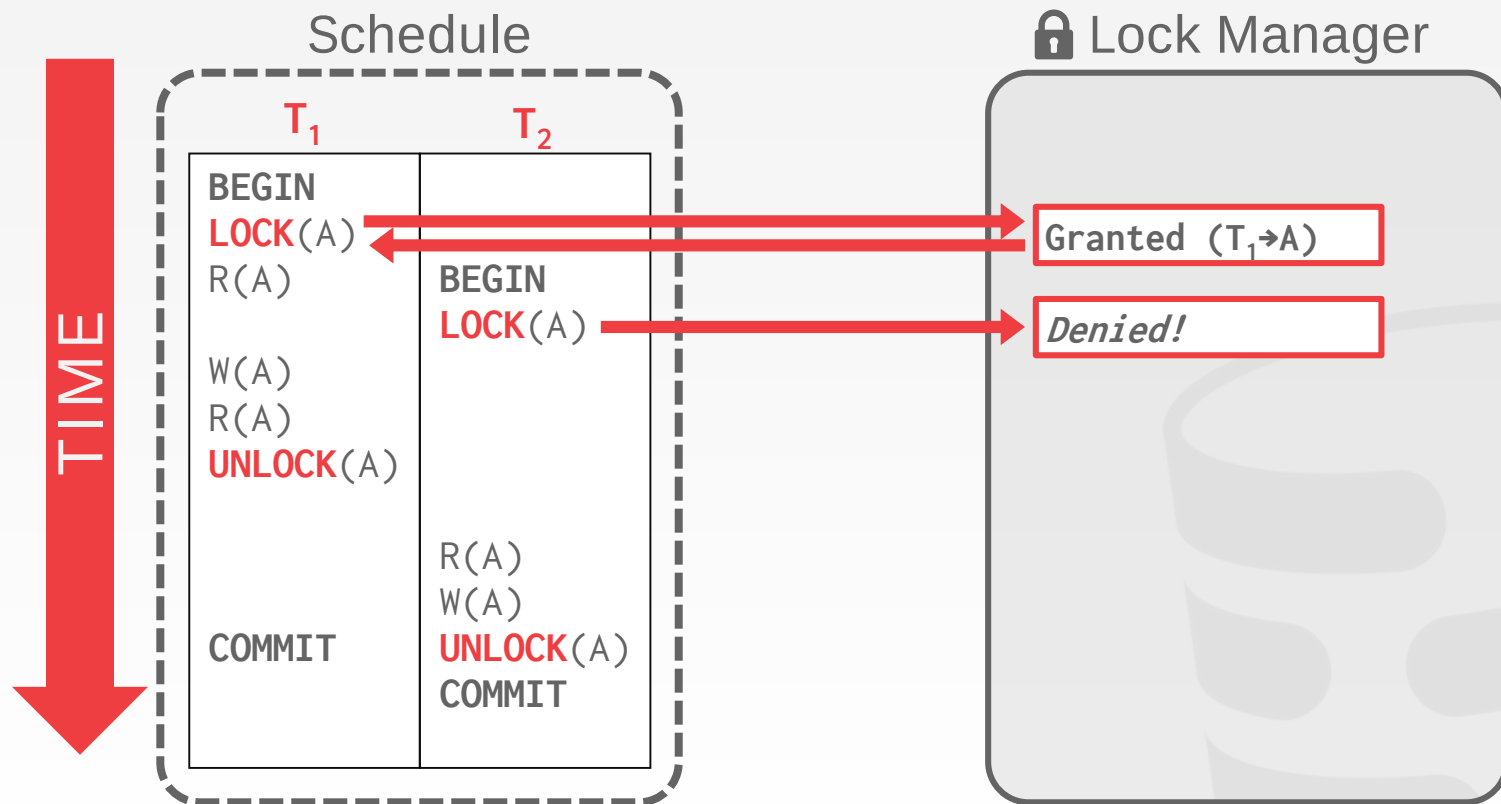
We need a way to guarantee that all execution schedules are correct (i.e., serializable) without knowing the entire schedule ahead of time.

Solution: Use **locks** to protect database objects.

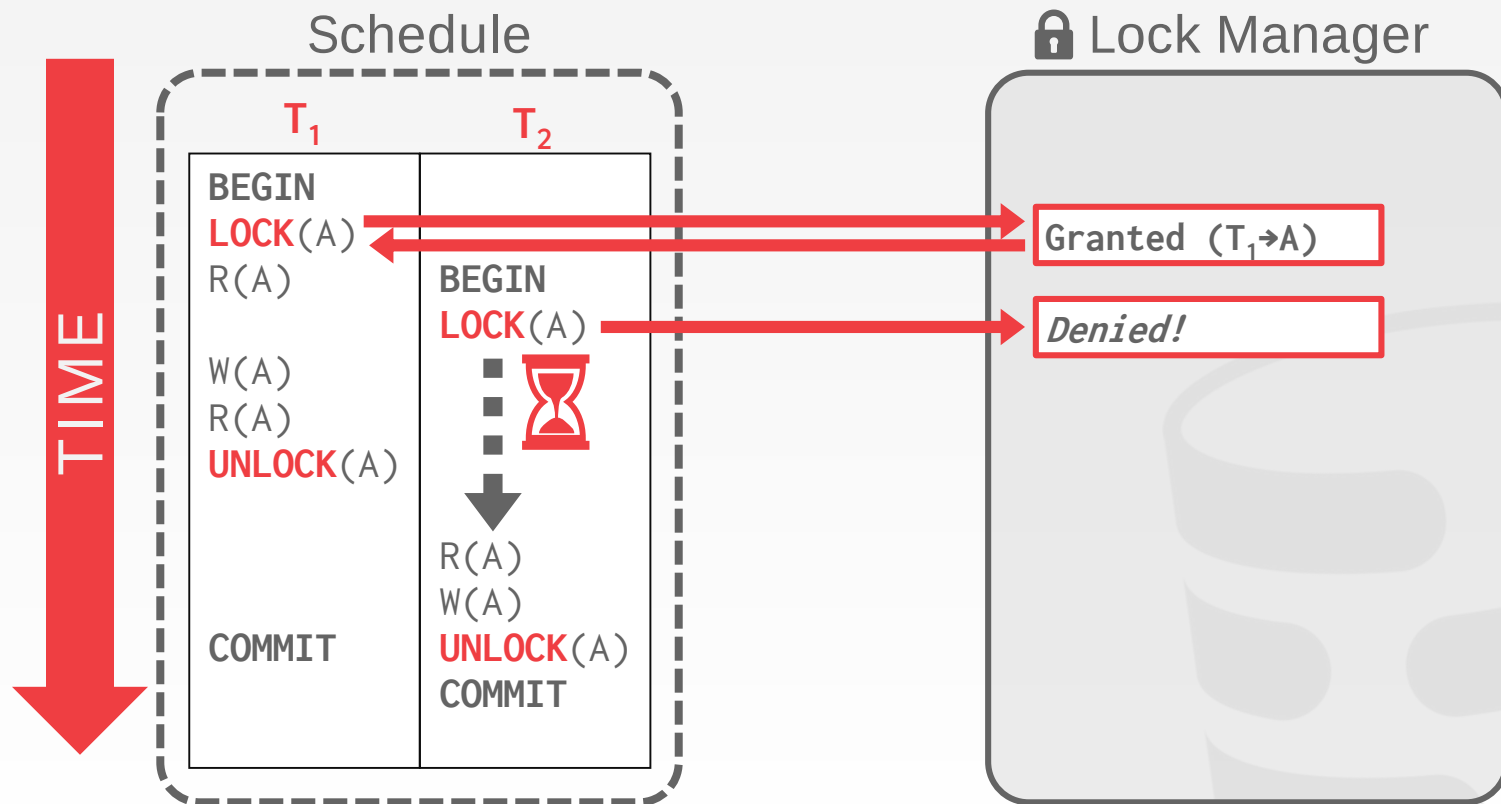
EXECUTING WITH LOCKS



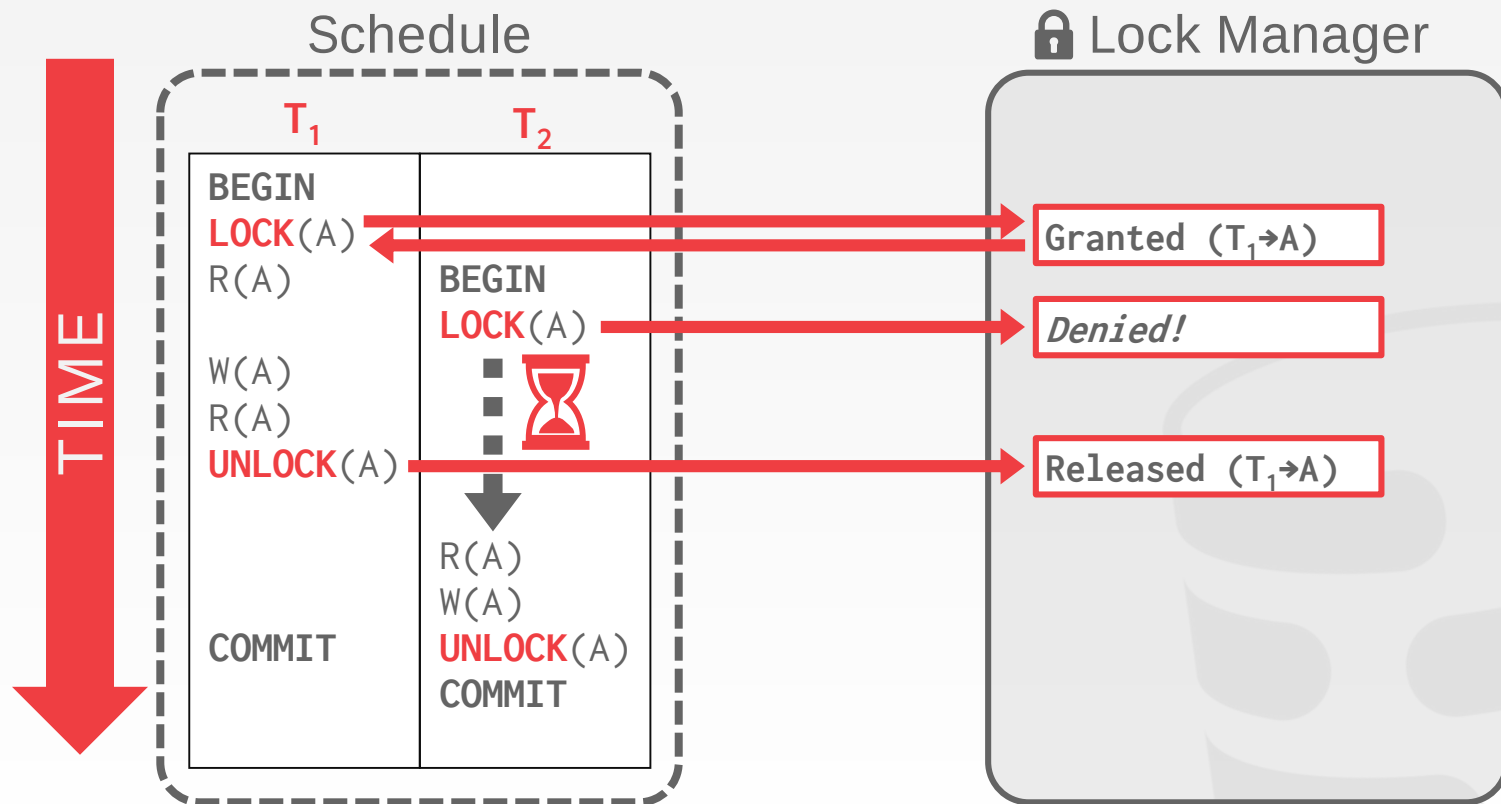
EXECUTING WITH LOCKS



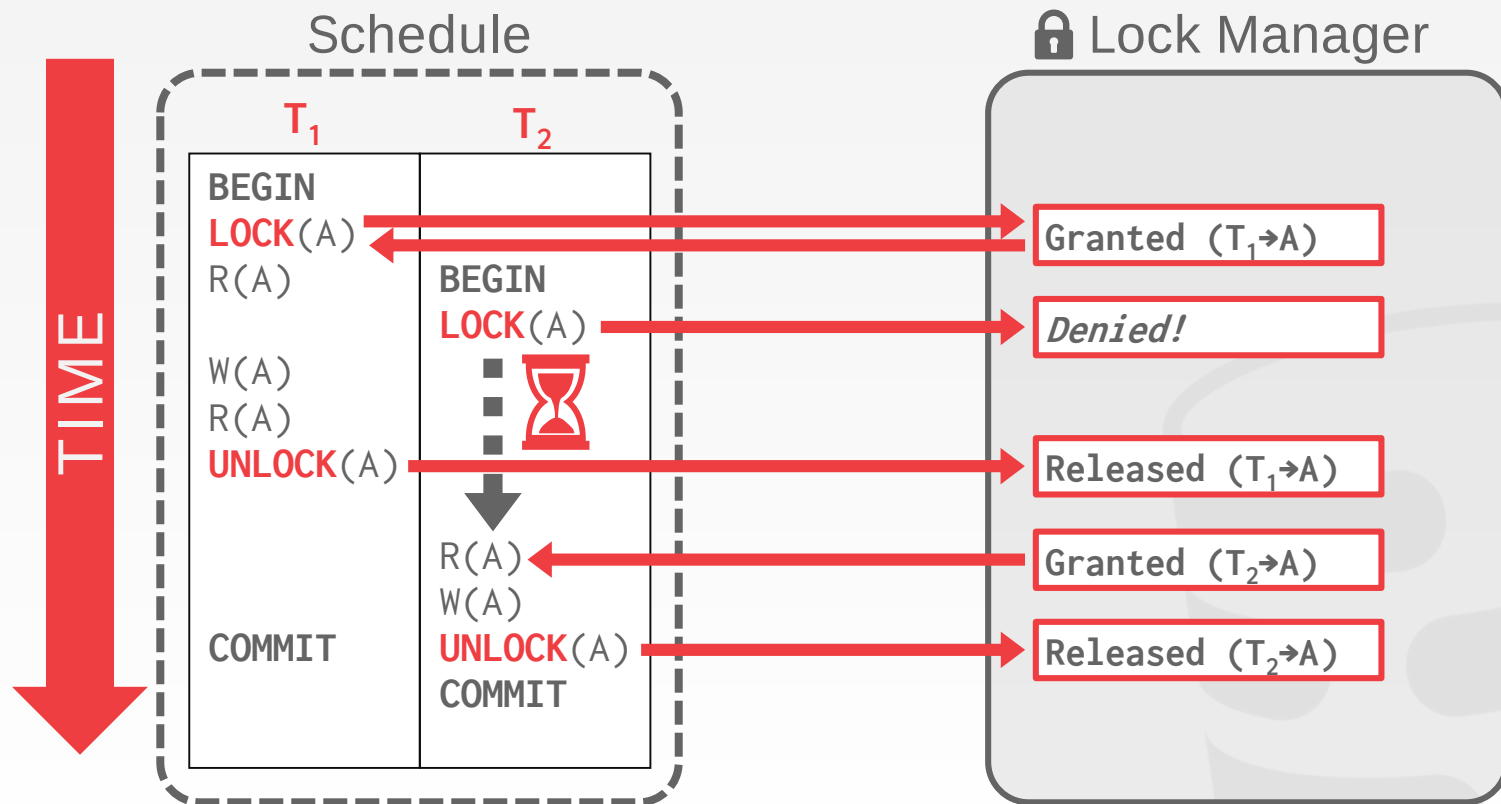
EXECUTING WITH LOCKS



EXECUTING WITH LOCKS



EXECUTING WITH LOCKS



TODAY'S AGENDA

Lock Types

Two-Phase Locking

Deadlock Detection + Prevention

Hierarchical Locking

Isolation Levels



LOCKS VS. LATCHES

	<i>Locks</i>	<i>Latches</i>
Separate...	User transactions	Threads
Protect...	Database Contents	In-Memory Data Structures
During...	Entire Transactions	Critical Sections
Modes...	Shared, Exclusive, Update, Intention	Read, Write
Deadlock	Detection & Resolution	Avoidance
...by...	Waits-for, Timeout, Aborts	Coding Discipline
Kept in...	Lock Manager	Protected Data Structure

Source: [Goetz Graefe](#)

BASIC LOCK TYPES

S-LOCK: Shared locks for reads.

X-LOCK: Exclusive locks for writes.

Compatibility Matrix		
	Shared	Exclusive
Shared	✓	X
Exclusive	X	X

EXECUTING WITH LOCKS

Transactions request locks (or upgrades).

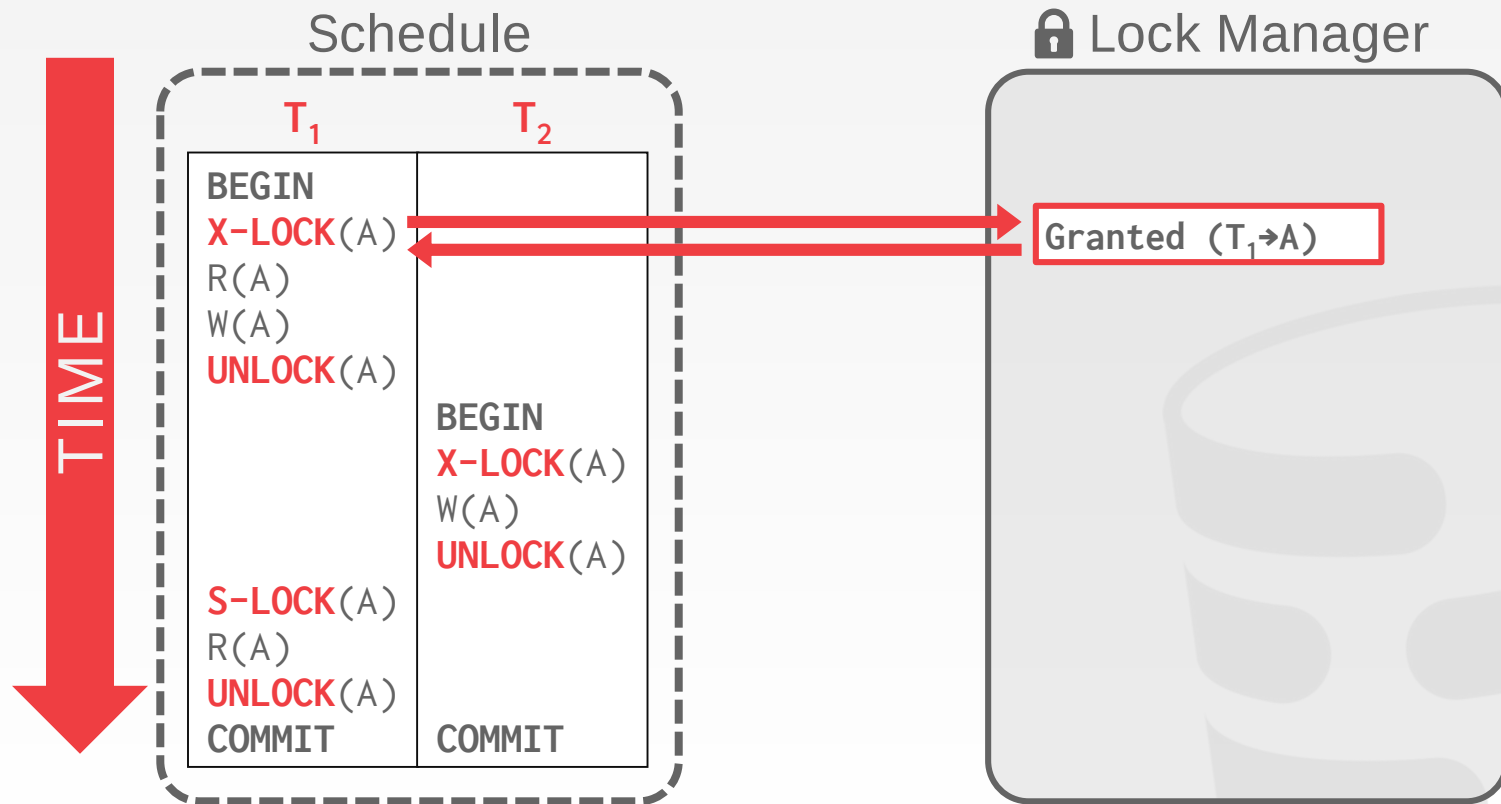
Lock manager grants or blocks requests.

Transactions release locks.

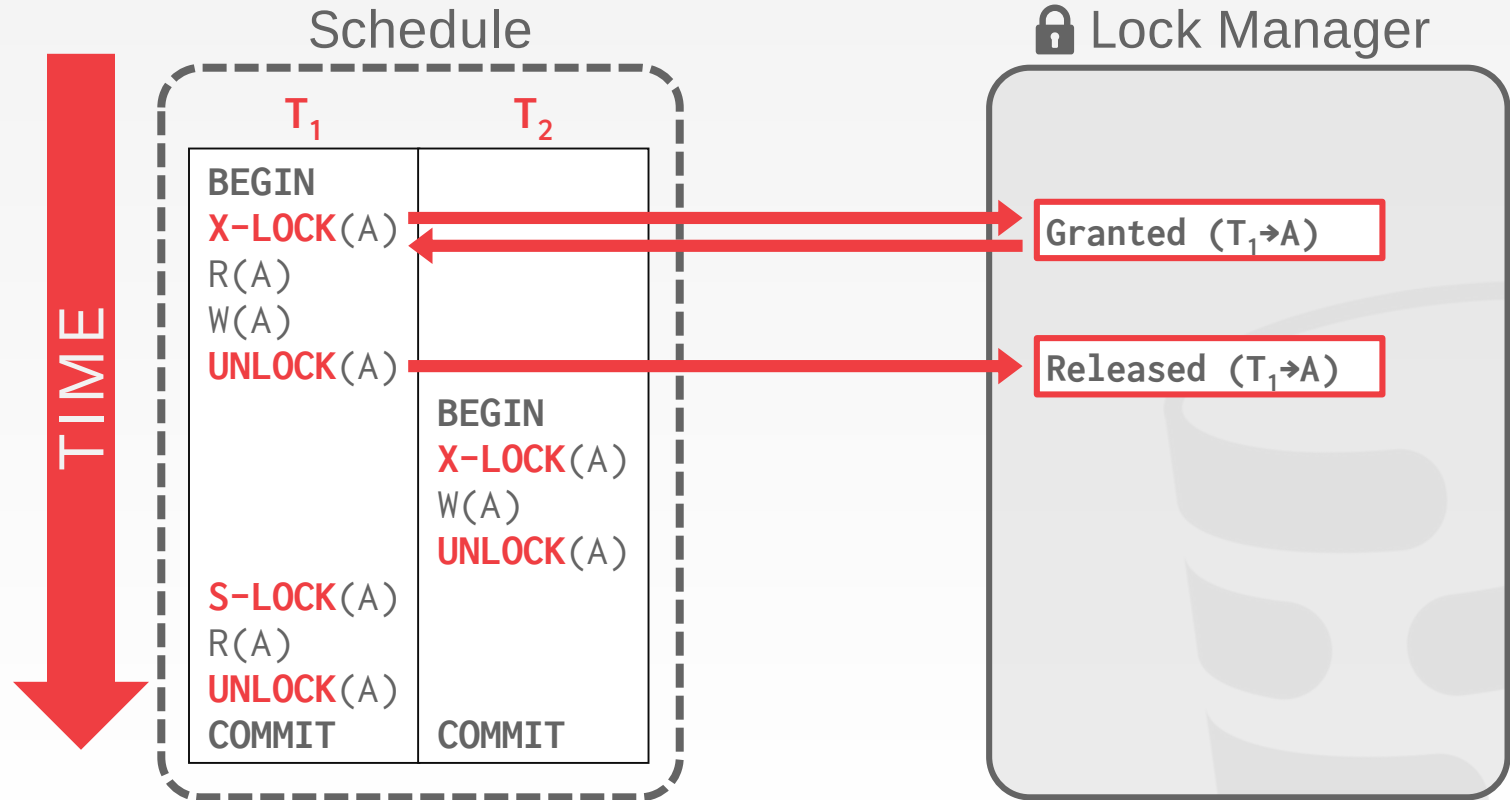
Lock manager updates its internal lock-table.

→ It keeps track of what transactions hold what locks and what transactions are waiting to acquire any locks.

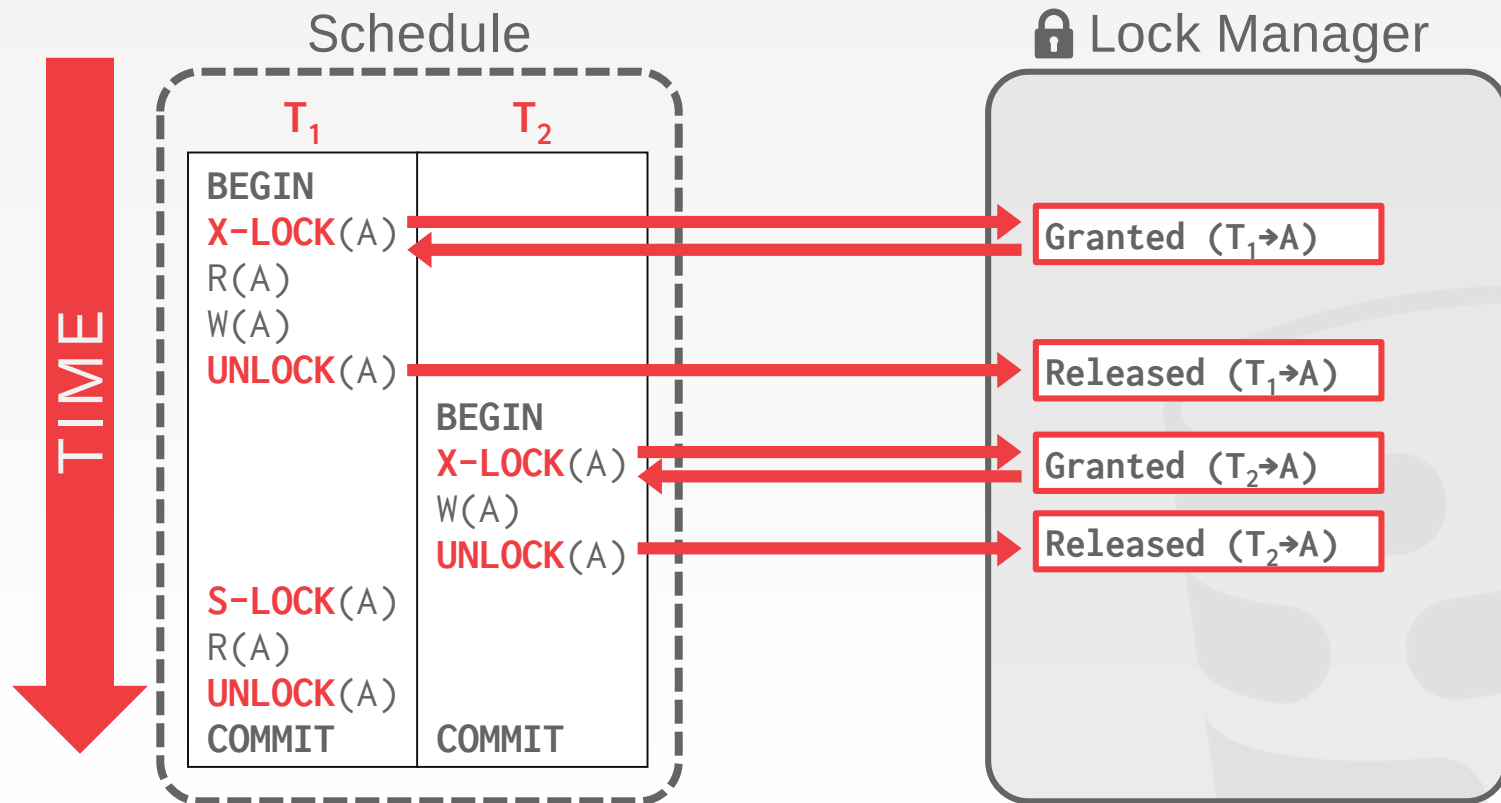
EXECUTING WITH LOCKS



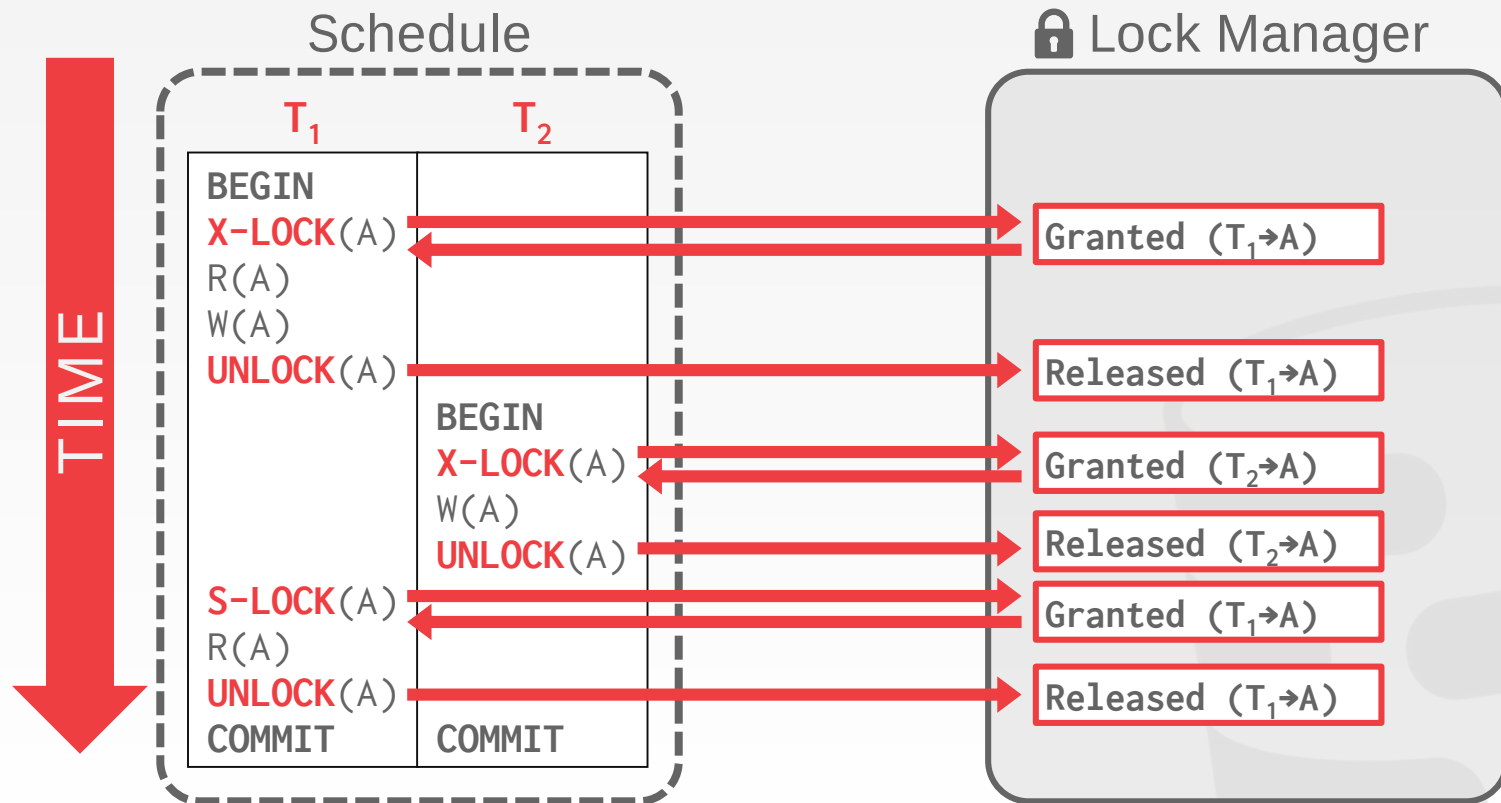
EXECUTING WITH LOCKS



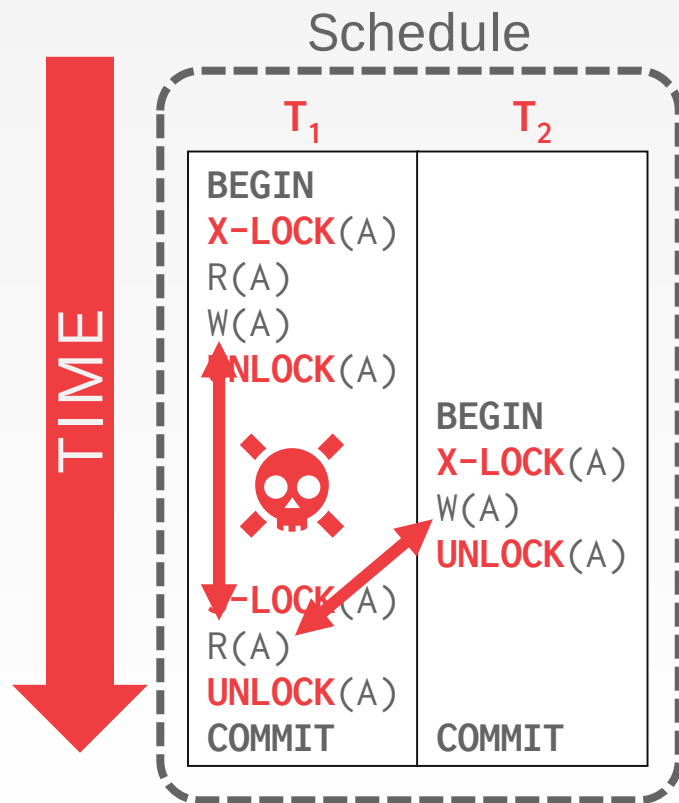
EXECUTING WITH LOCKS



EXECUTING WITH LOCKS



EXECUTING WITH LOCKS



Lock Manager

Granted ($T_1 \rightarrow A$)

Released ($T_1 \rightarrow A$)

Granted ($T_2 \rightarrow A$)

Released ($T_2 \rightarrow A$)

Granted ($T_1 \rightarrow A$)

Released ($T_1 \rightarrow A$)

CONCURRENCY CONTROL PROTOCOL

Two-phase locking (2PL) is a concurrency control protocol that determines whether a txn can access an object in the database on the fly.

The protocol does not need to know all the queries that a txn will execute ahead of time.

TWO-PHASE LOCKING

Phase #1: Growing

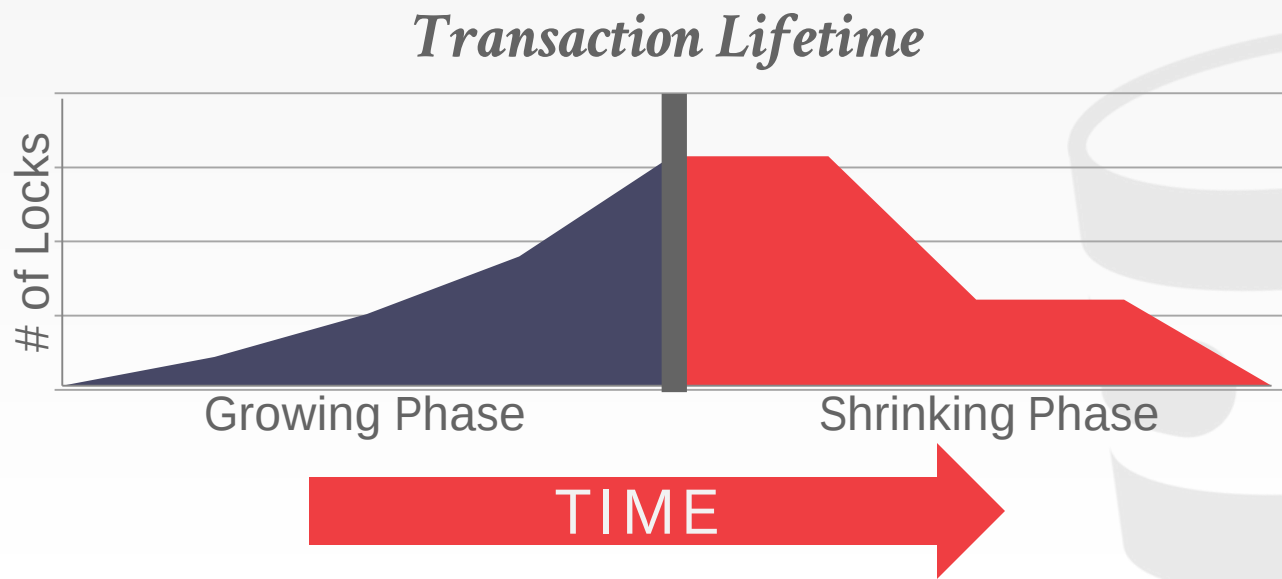
- Each txn requests the locks that it needs from the DBMS's lock manager.
- The lock manager grants/denies lock requests.

Phase #2: Shrinking

- The txn is allowed to only release locks that it previously acquired. It cannot acquire new locks.

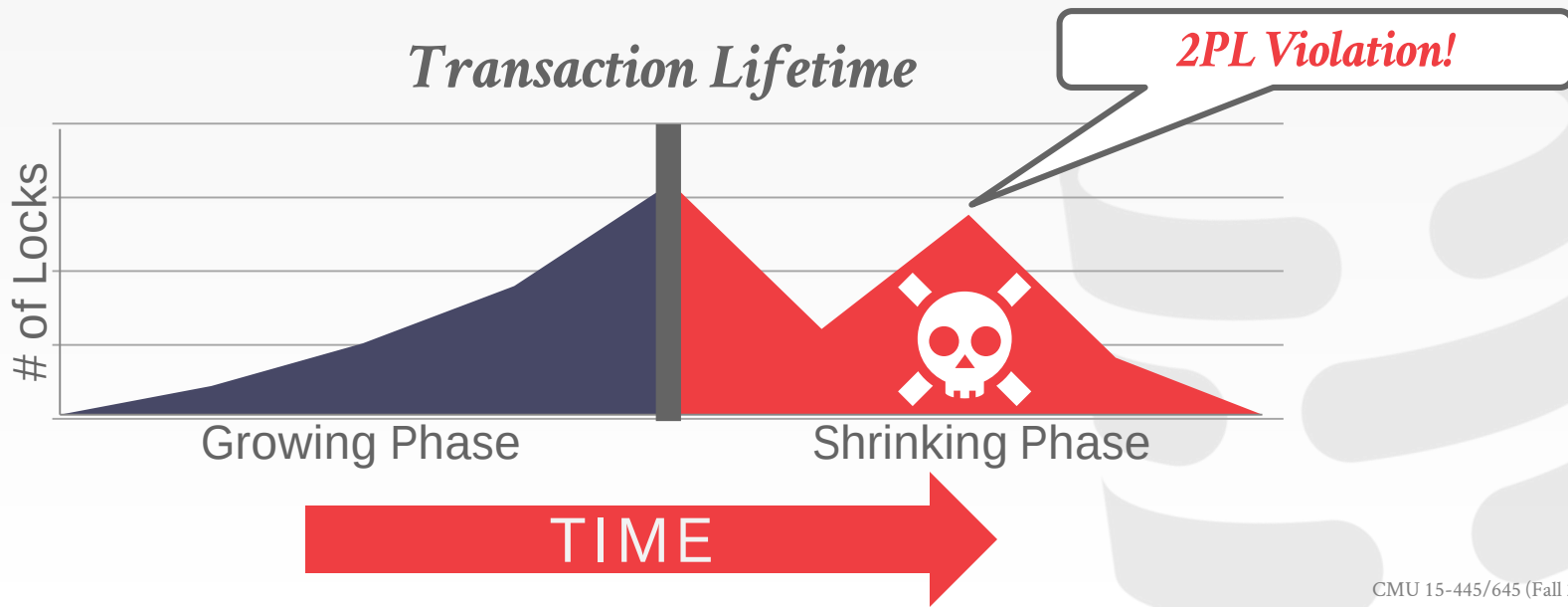
TWO-PHASE LOCKING

The txn is not allowed to acquire/upgrade locks after the growing phase finishes.

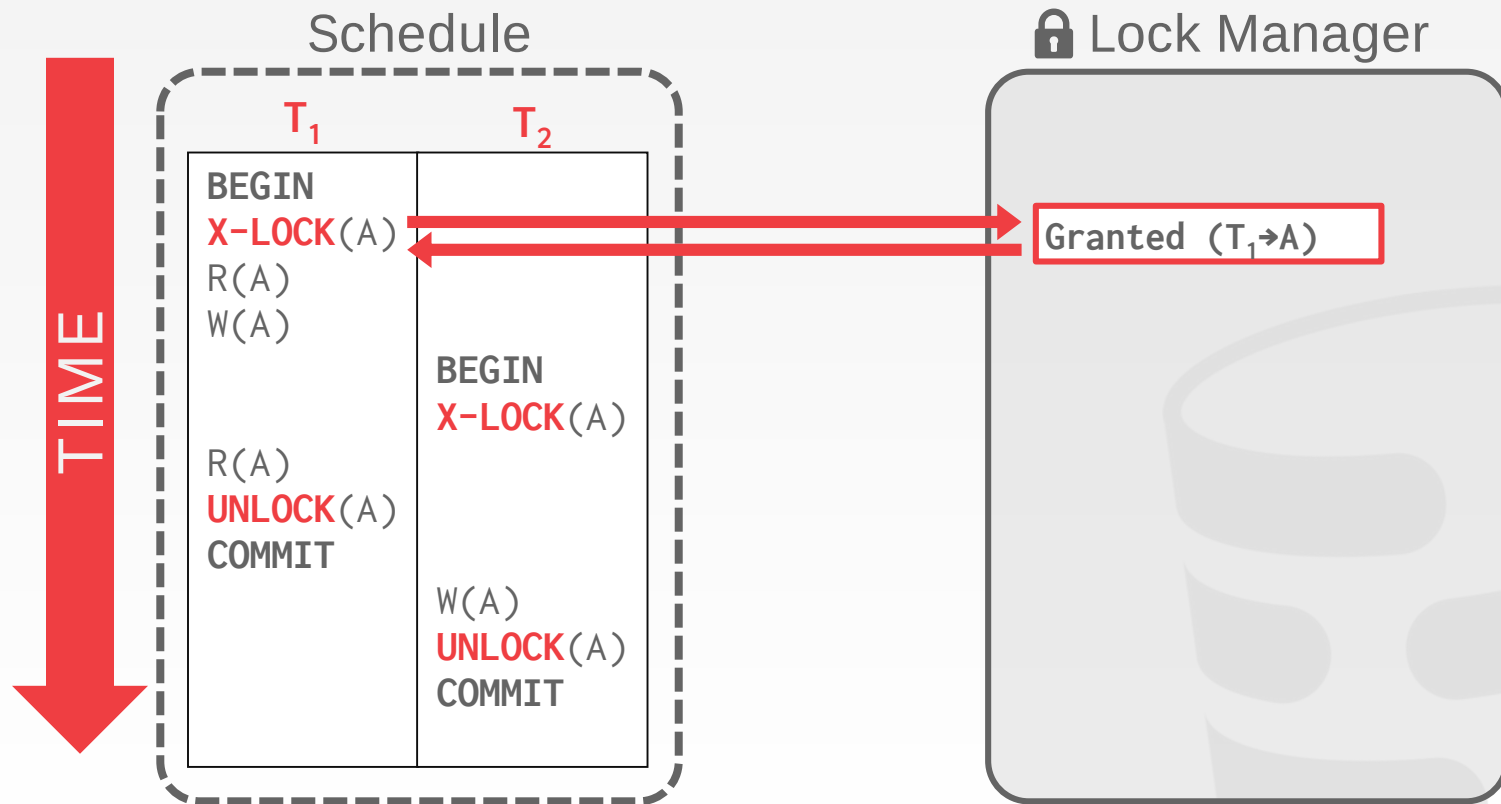


TWO-PHASE LOCKING

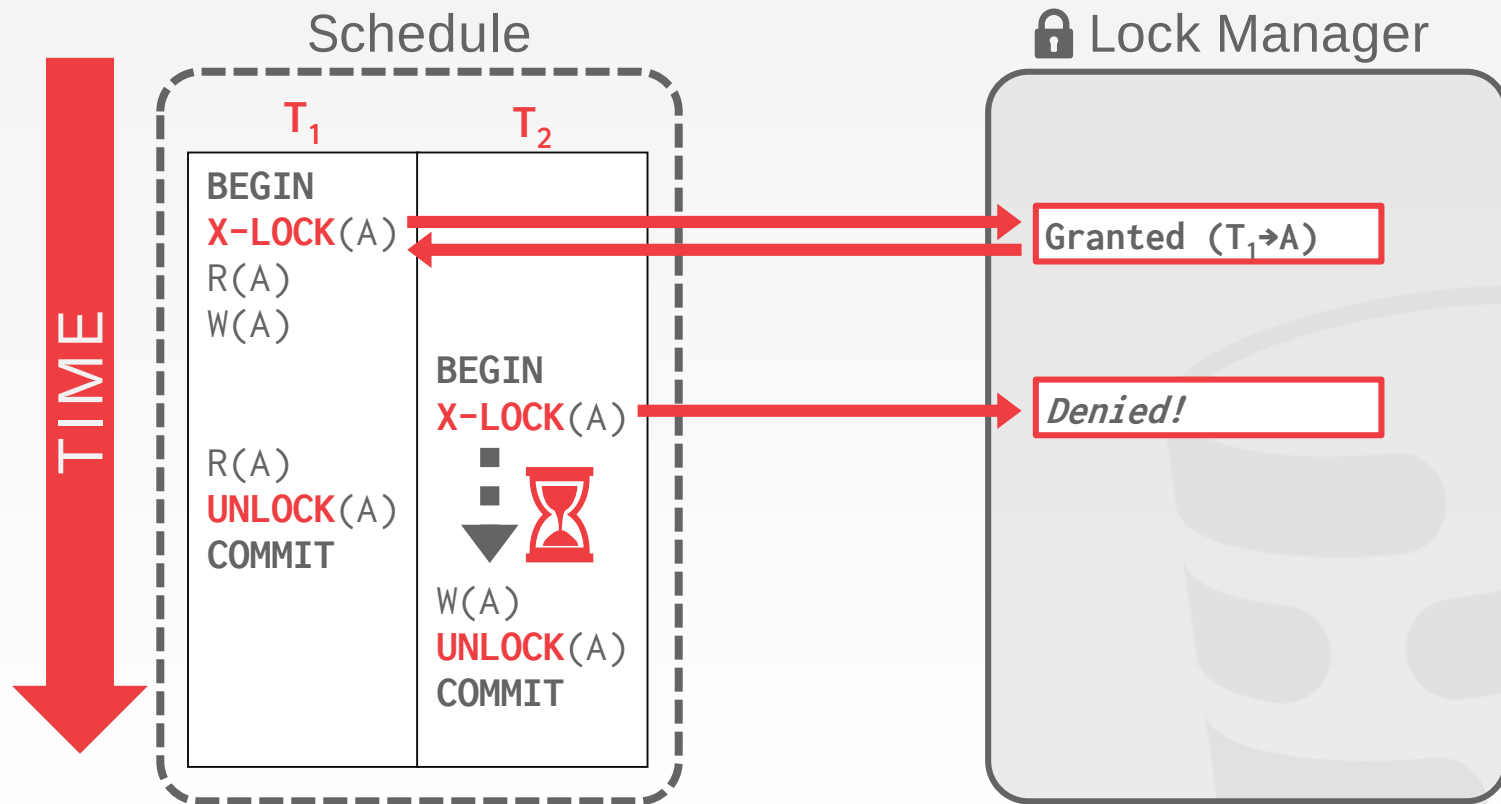
The txn is not allowed to acquire/upgrade locks after the growing phase finishes.



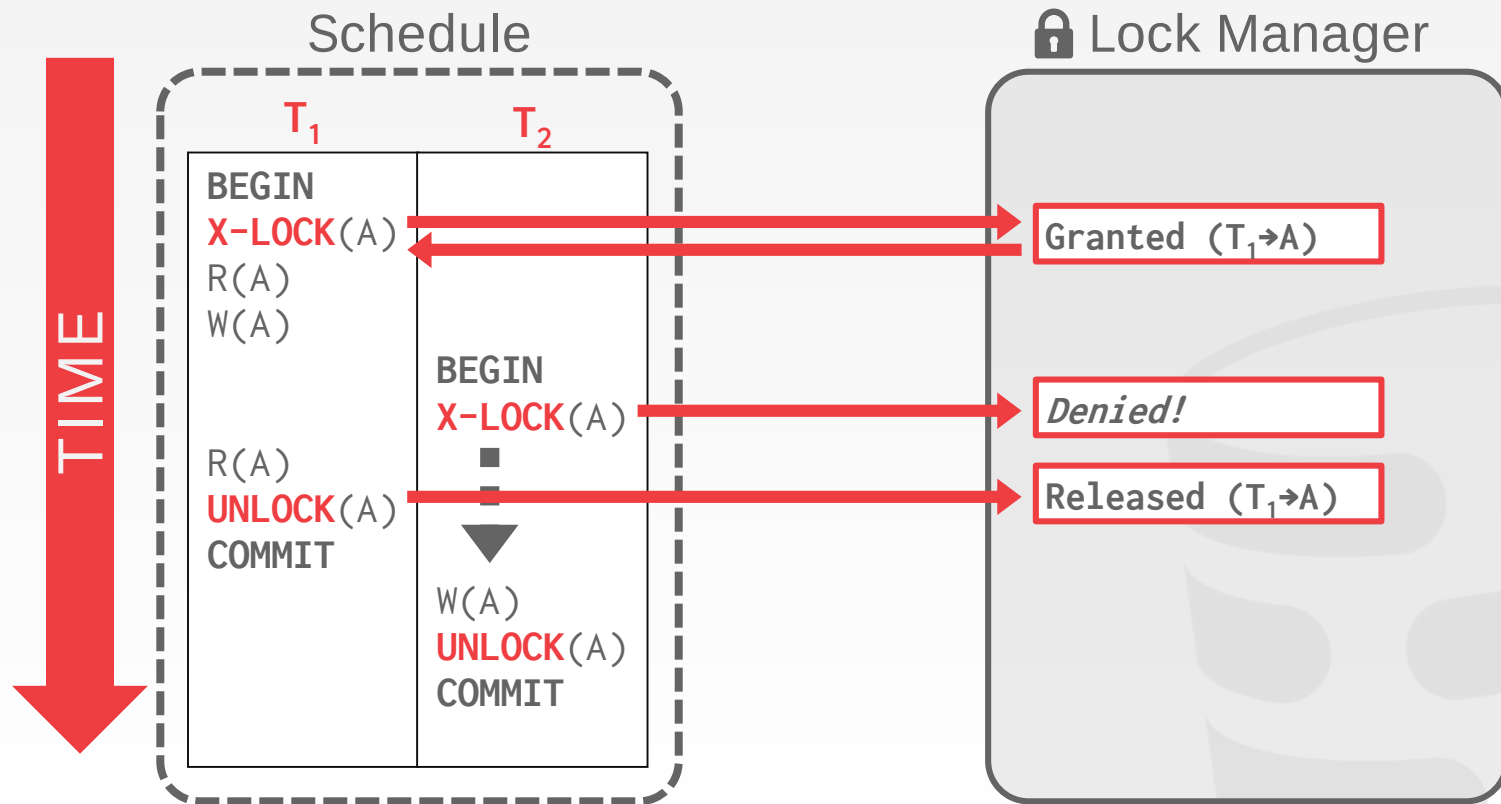
EXECUTING WITH 2PL



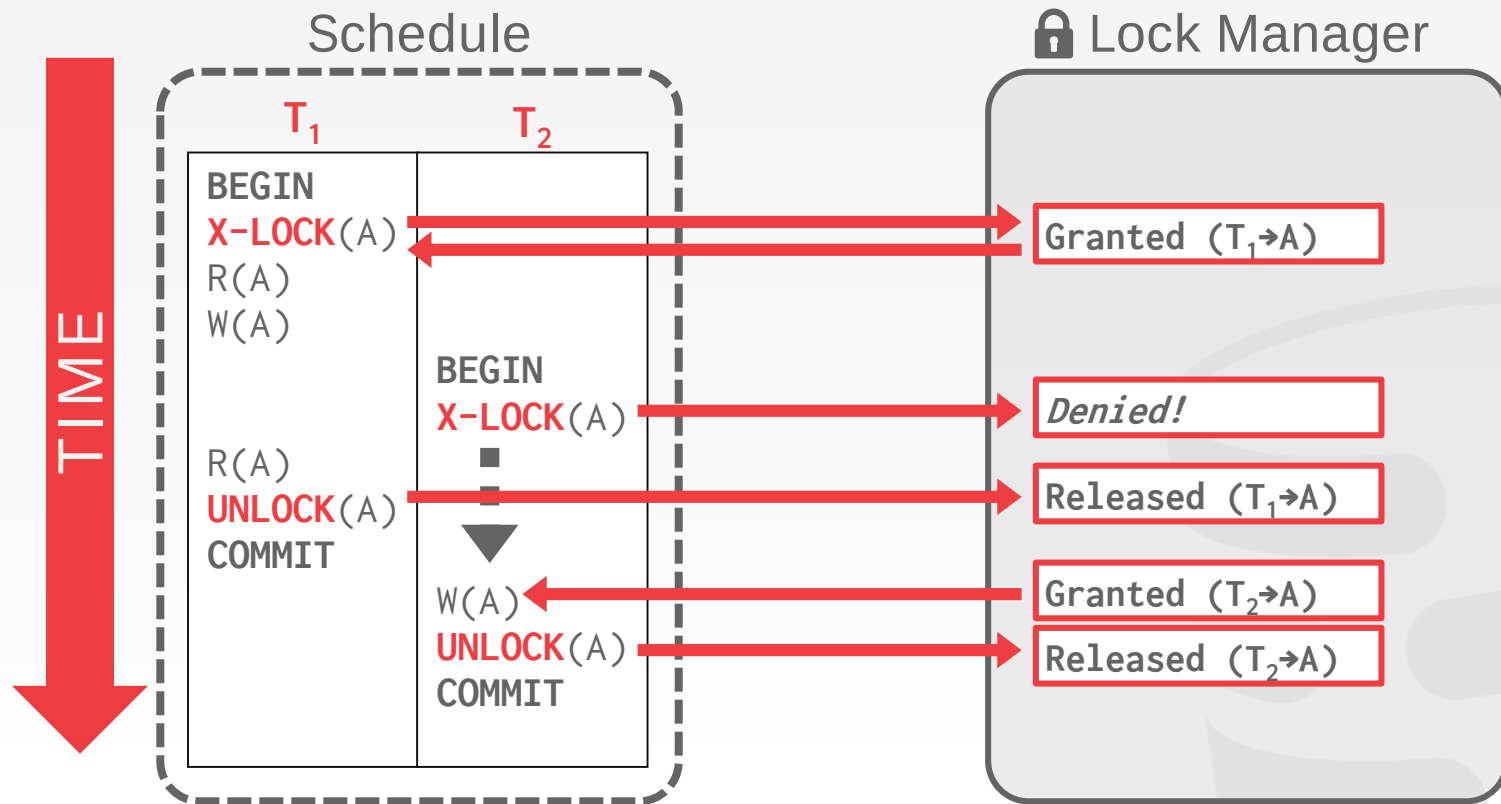
EXECUTING WITH 2PL



EXECUTING WITH 2PL



EXECUTING WITH 2PL



TWO-PHASE LOCKING

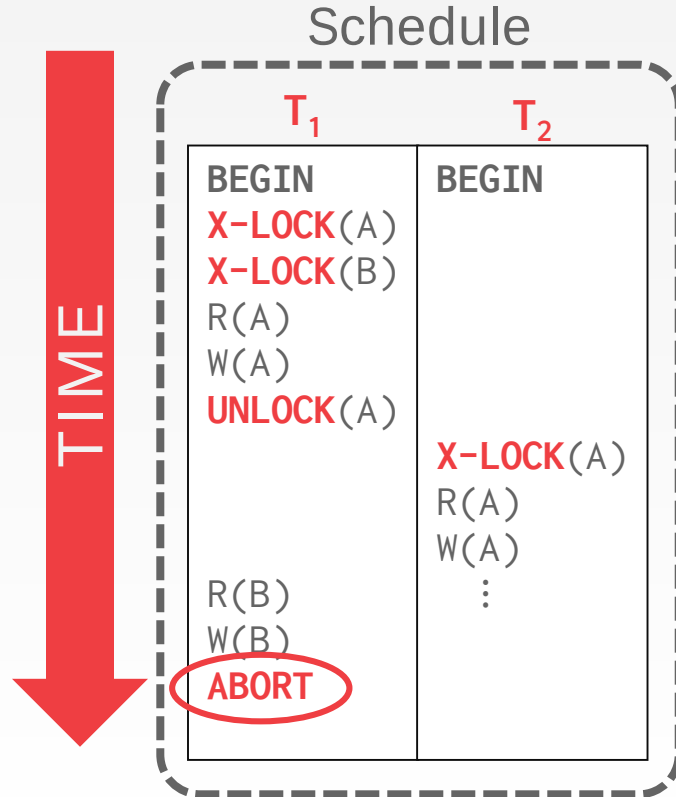
2PL on its own is sufficient to guarantee conflict serializability.

→ It generates schedules whose precedence graph is acyclic.

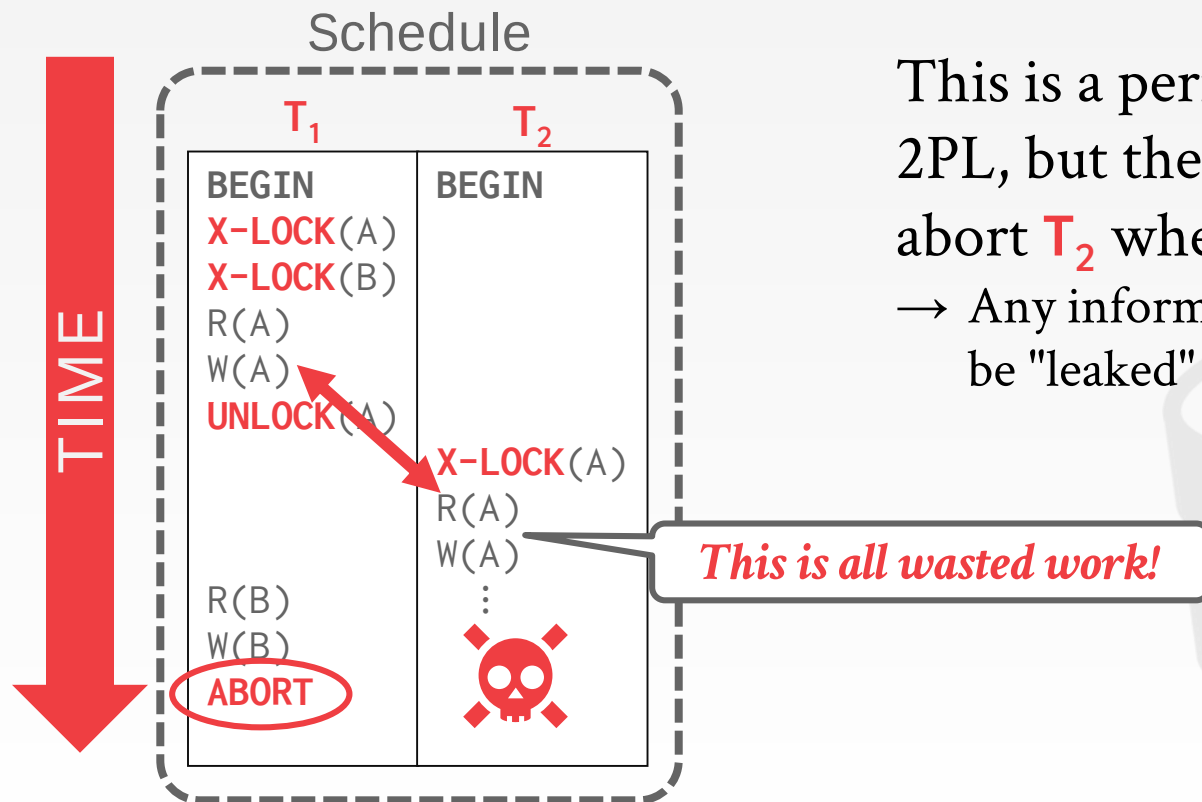
But it is subject to **cascading aborts**.



2PL – CASCADING ABORTS



2PL – CASCADING ABORTS



This is a permissible schedule in 2PL, but the DBMS has to also abort T_2 when T_1 aborts.

→ Any information about T_1 cannot be "leaked" to the outside world.

2PL OBSERVATIONS

There are potential schedules that are serializable but would not be allowed by 2PL.

→ Locking limits concurrency.

May still have "dirty reads".

→ Solution: **Strong Strict 2PL (aka Rigorous 2PL)**

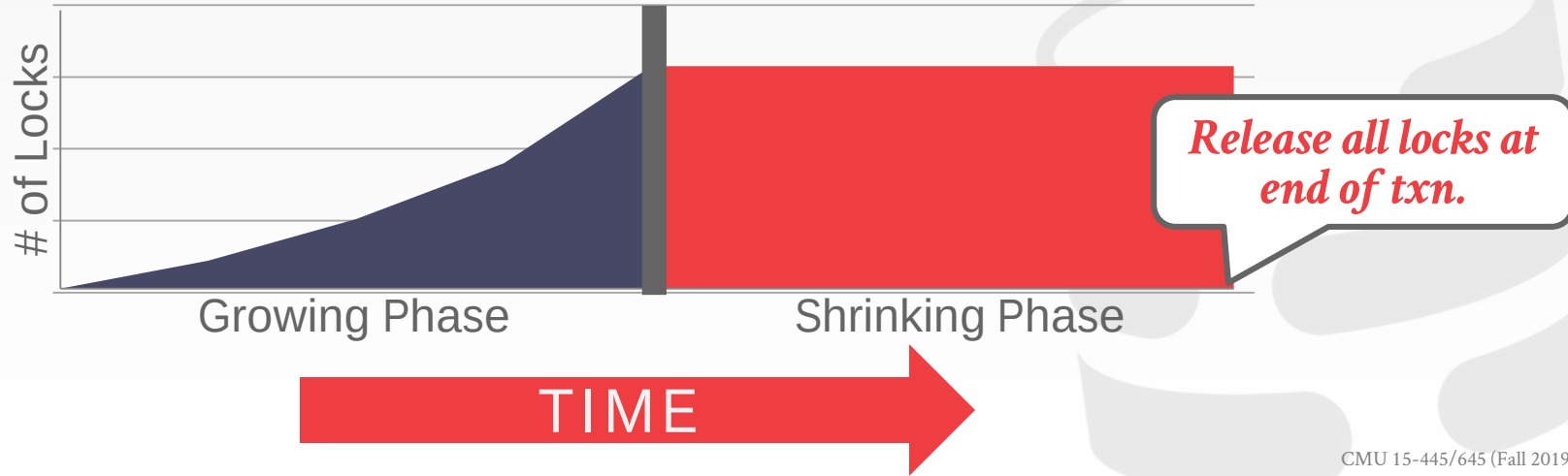
May lead to deadlocks.

→ Solution: **Detection** or **Prevention**

STRONG STRICT TWO-PHASE LOCKING

The txn is not allowed to acquire/upgrade locks after the growing phase finishes.

Allows only conflict serializable schedules, but it is often stronger than needed for some apps.



STRONG STRICT TWO-PHASE LOCKING

A schedule is **strict** if a value written by a txn is not read or overwritten by other txns until that txn finishes.

Advantages:

- Does not incur cascading aborts.
- Aborted txns can be undone by just restoring original values of modified tuples.

EXAMPLES

T₁ – Move \$100 from Andy's account (**A**) to his bookie's account (**B**).

T₂ – Compute the total amount in all accounts and return it to the application.

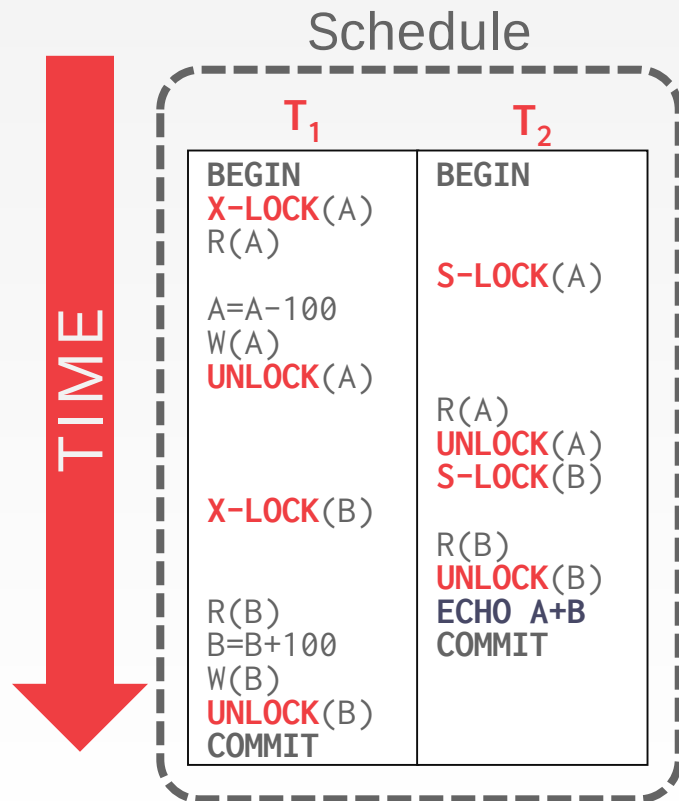
T₁

```
BEGIN
A=A-100
B=B+100
COMMIT
```

T₂

```
BEGIN
ECHO A+B
COMMIT
```

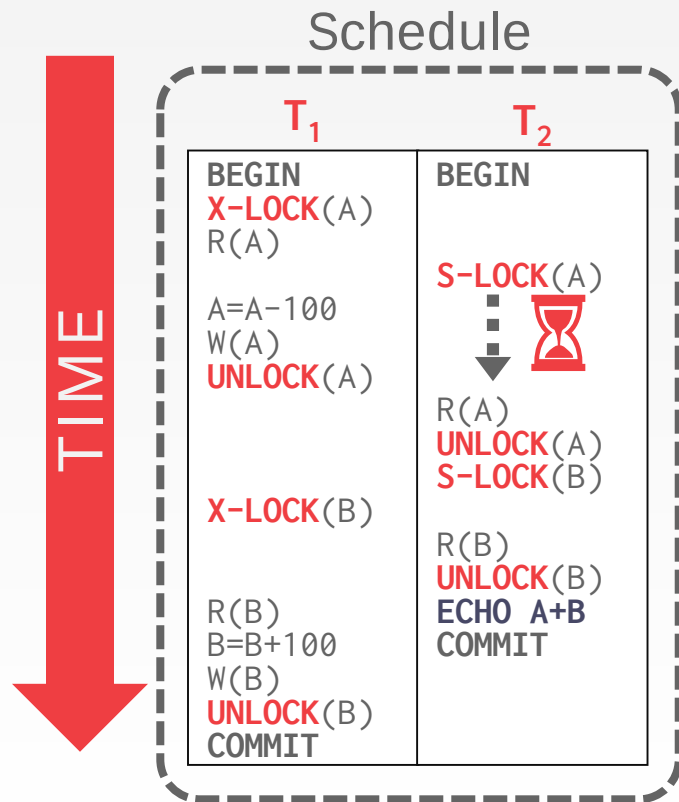
NON-2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

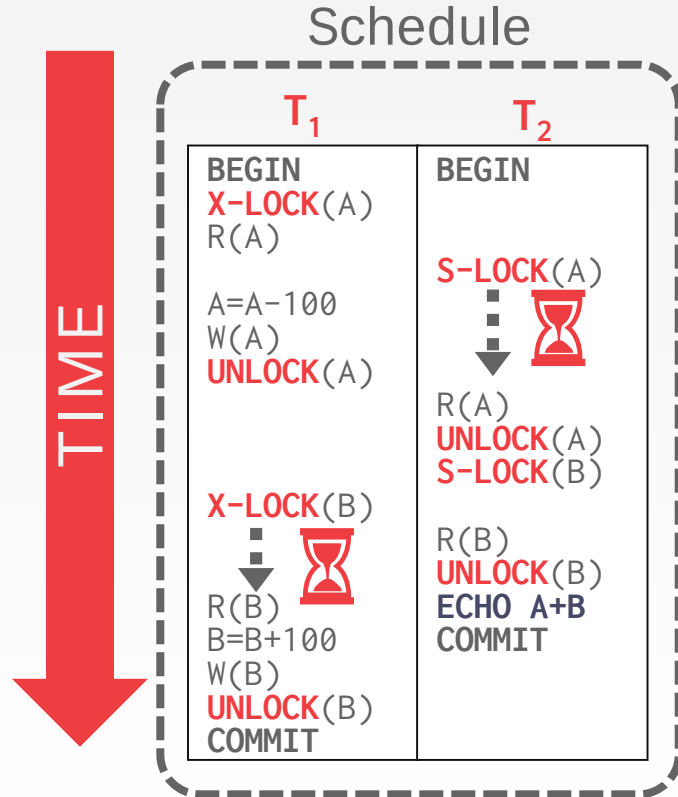
NON-2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

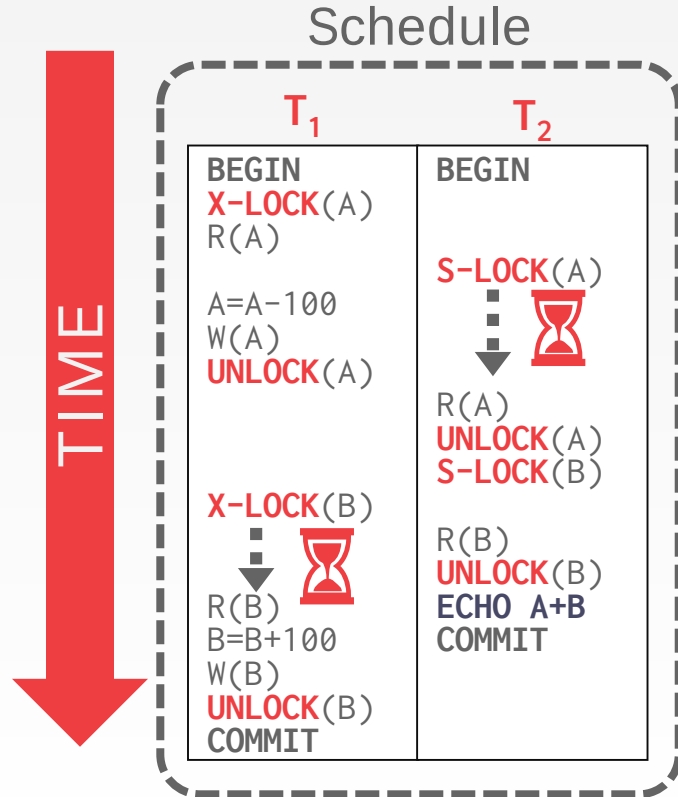
NON-2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

NON-2PL EXAMPLE



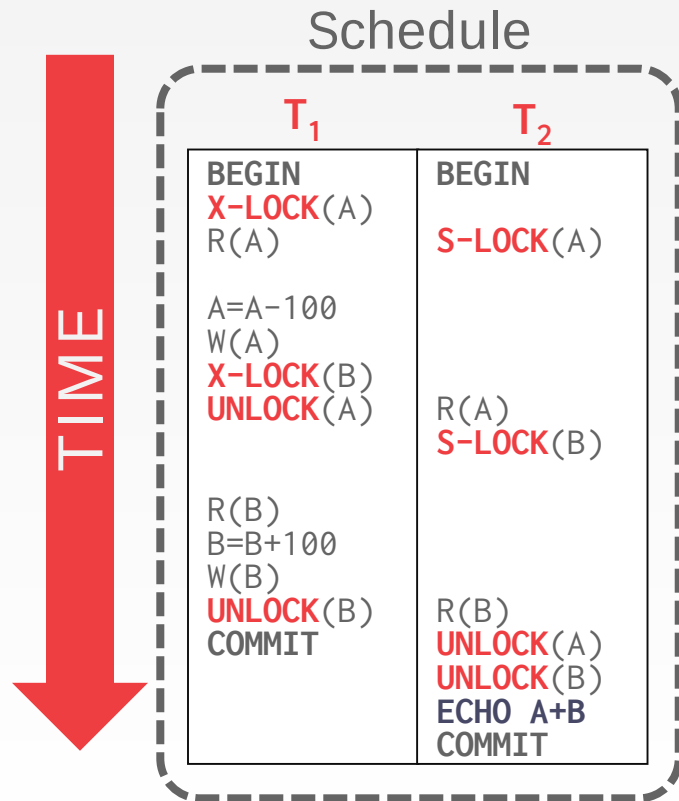
Initial Database State

A=1000, **B**=1000

T_2 Output

A+B=1100

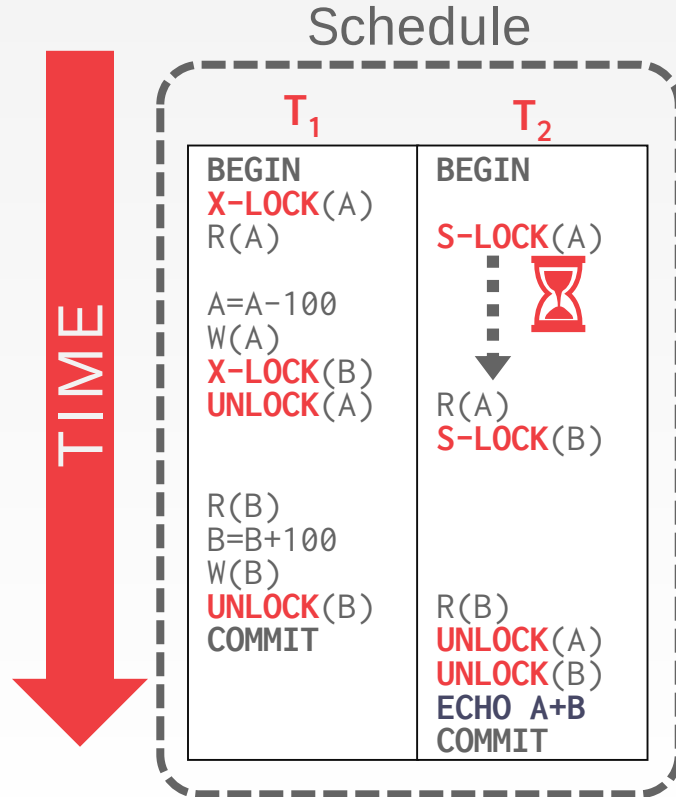
2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

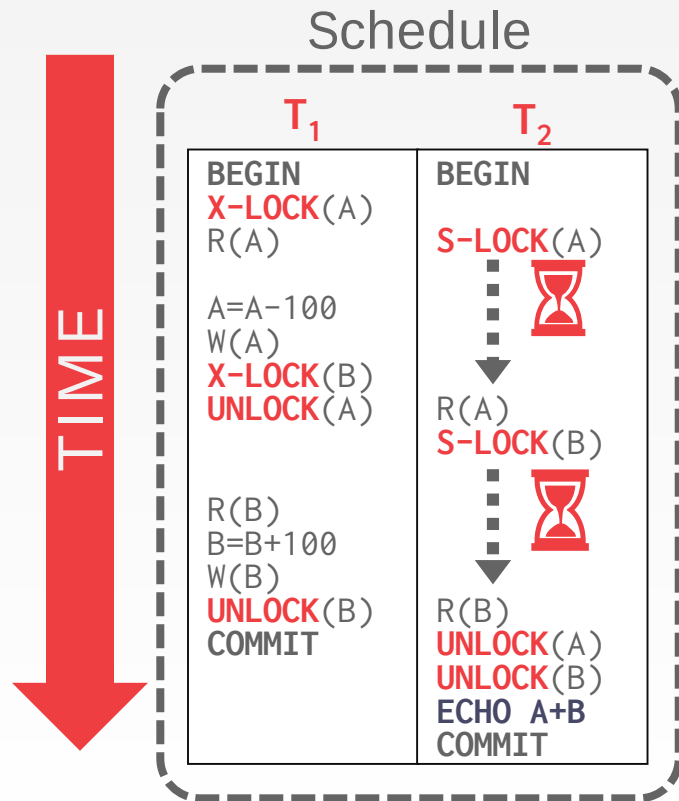
2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

2PL EXAMPLE



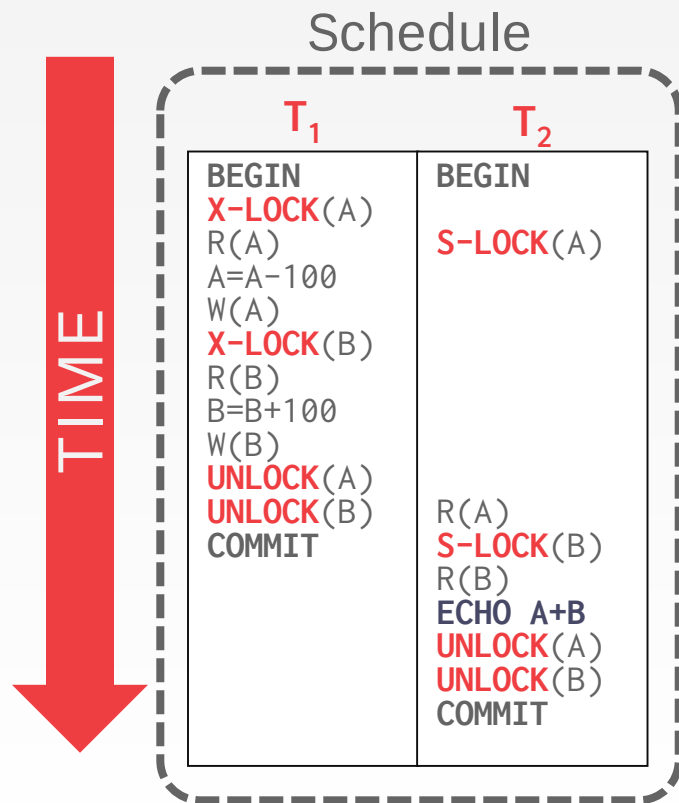
Initial Database State

A=1000, **B**=1000

T_2 Output

A+B=2000

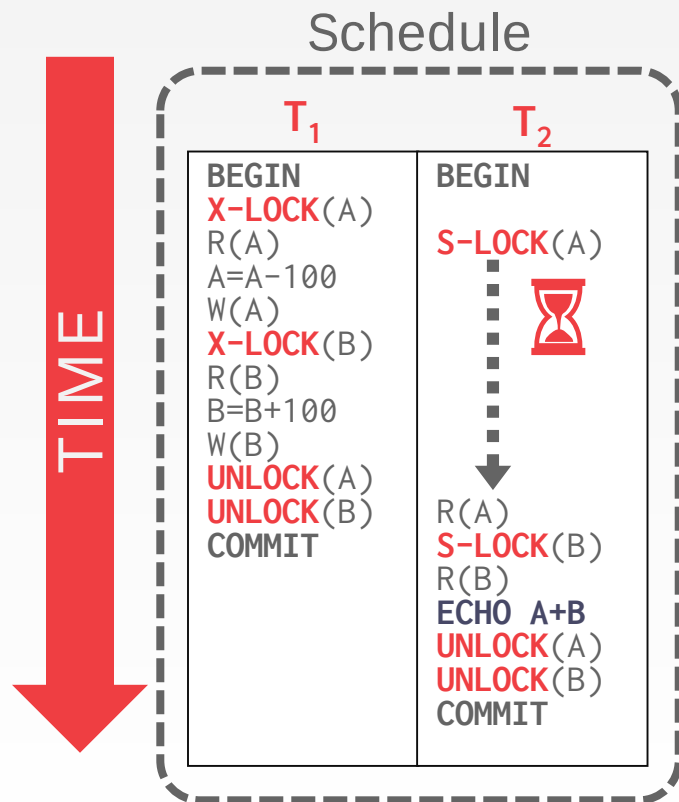
STRONG STRICT 2PL EXAMPLE



Initial Database State

A=1000, **B**=1000

STRONG STRICT 2PL EXAMPLE



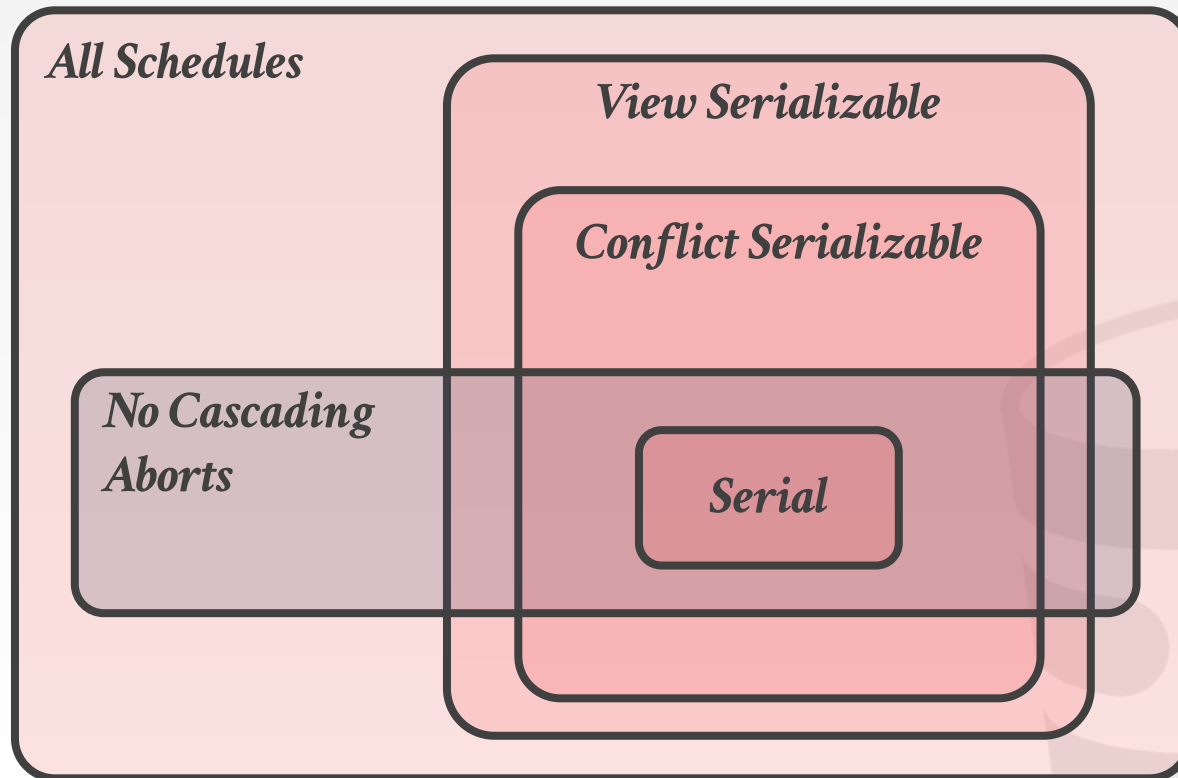
Initial Database State

A=1000, **B**=1000

T_2 Output

A+B=2000

UNIVERSE OF SCHEDULES



2PL OBSERVATIONS

There are potential schedules that are serializable but would not be allowed by 2PL.

→ Locking limits concurrency.

May still have "dirty reads".

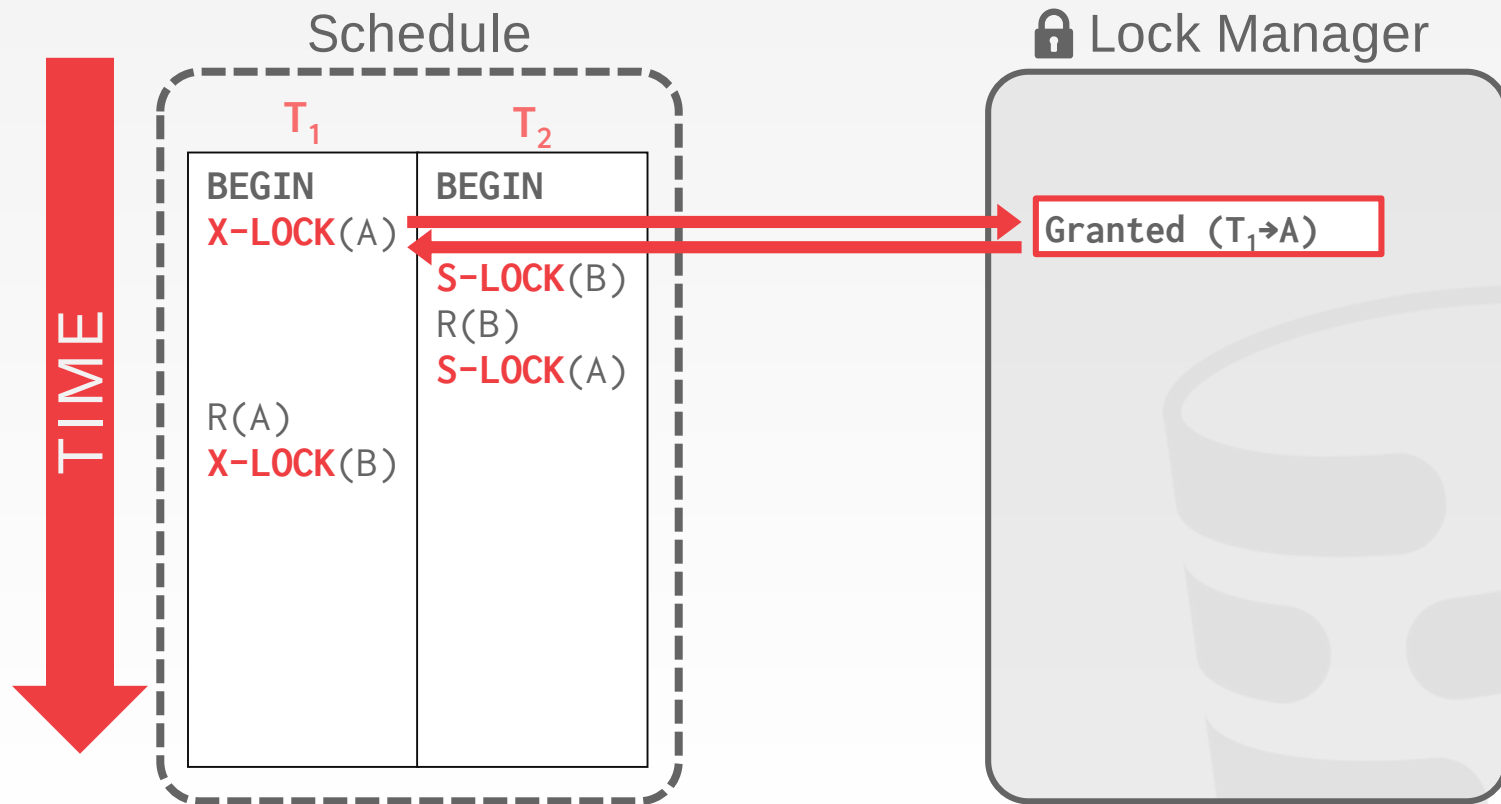
→ Solution: **Strong Strict 2PL (Rigorous)**

May lead to deadlocks.

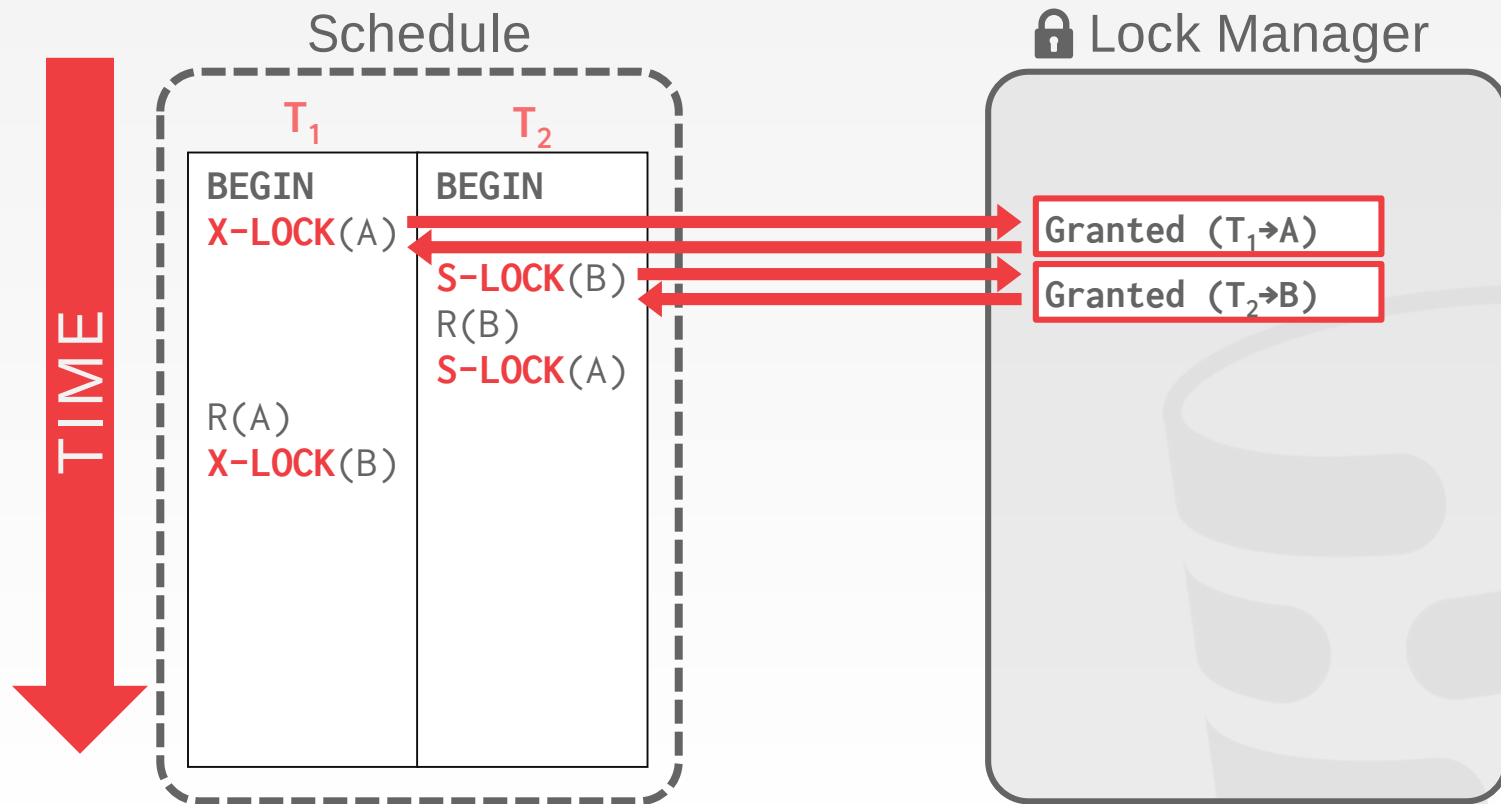
→ Solution: **Detection** or **Prevention**



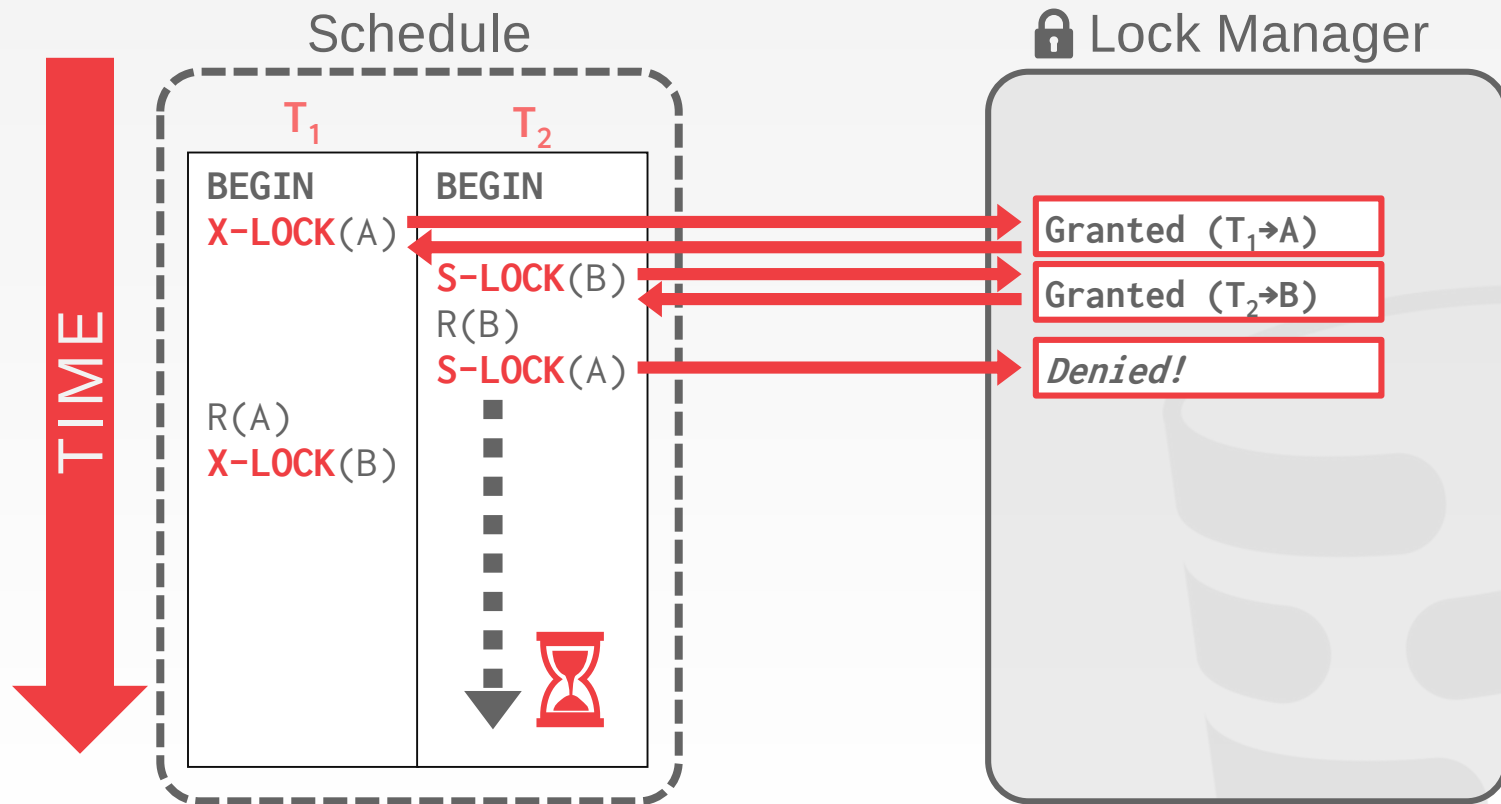
SHIT JUST GOT REAL, SON



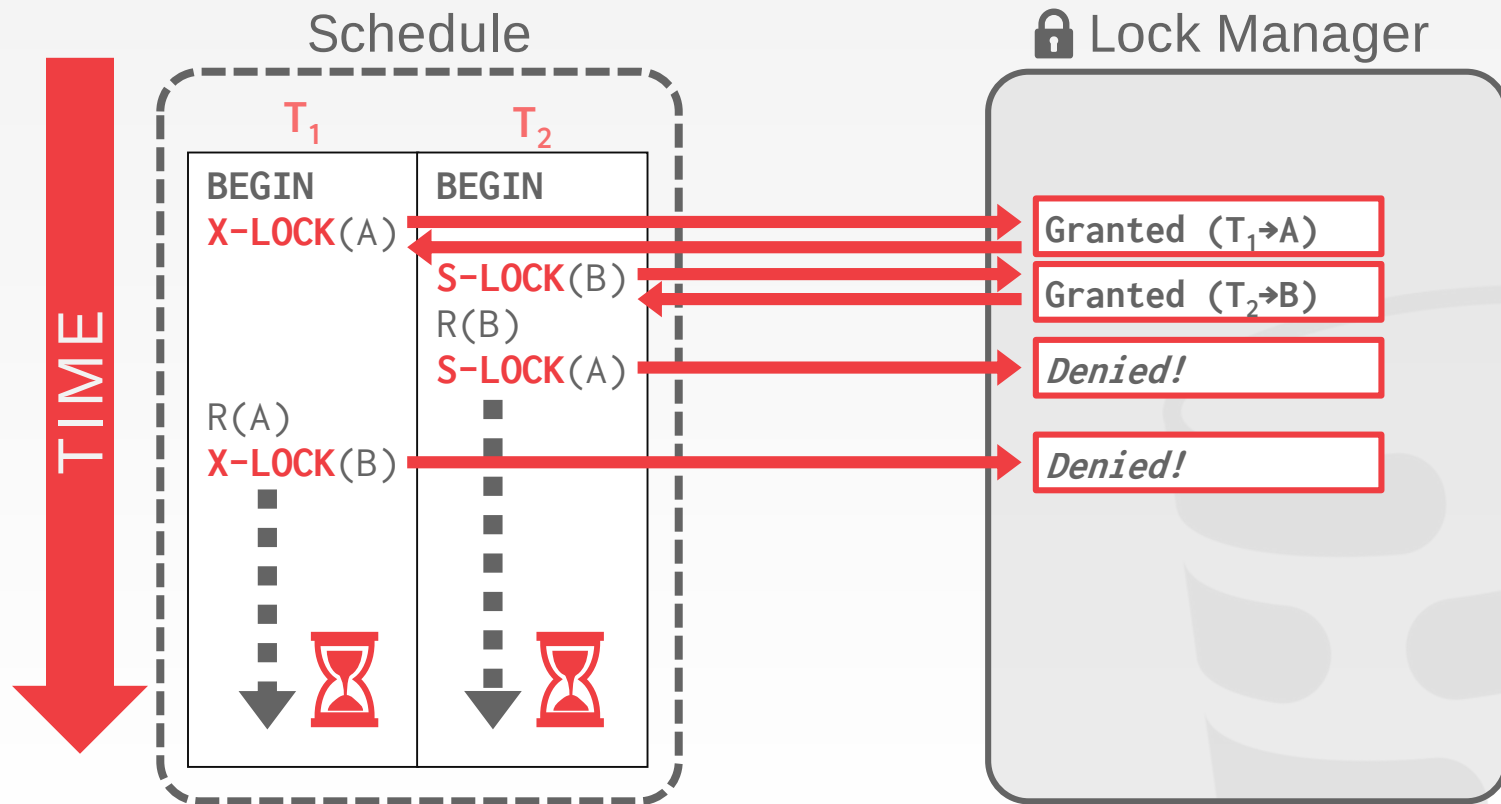
SHIT JUST GOT REAL, SON



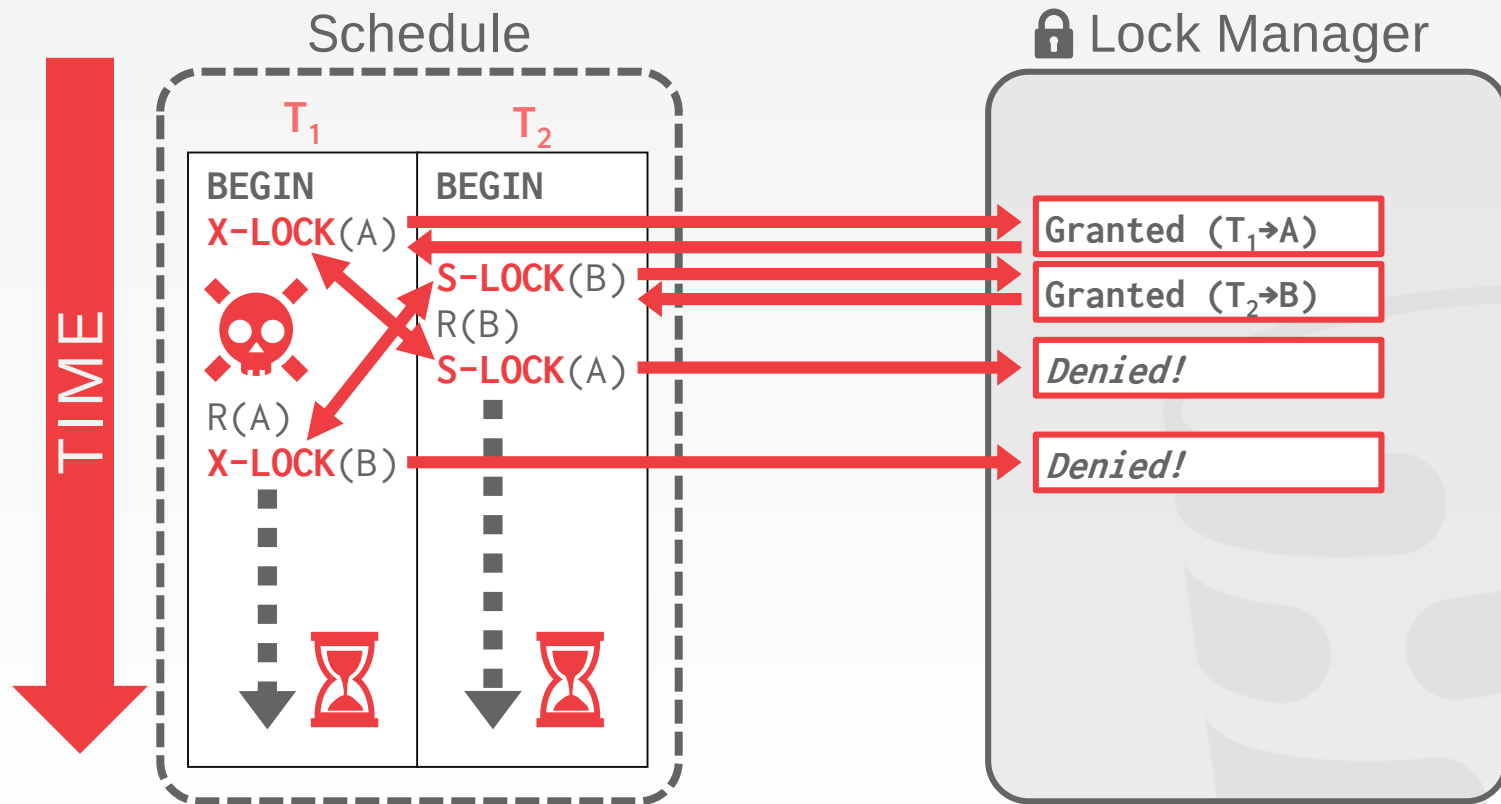
SHIT JUST GOT REAL, SON



SHIT JUST GOT REAL, SON



SHIT JUST GOT REAL, SON



2PL DEADLOCKS

A **deadlock** is a cycle of transactions waiting for locks to be released by each other.

Two ways of dealing with deadlocks:

- **Approach #1: Deadlock Detection**
- **Approach #2: Deadlock Prevention**



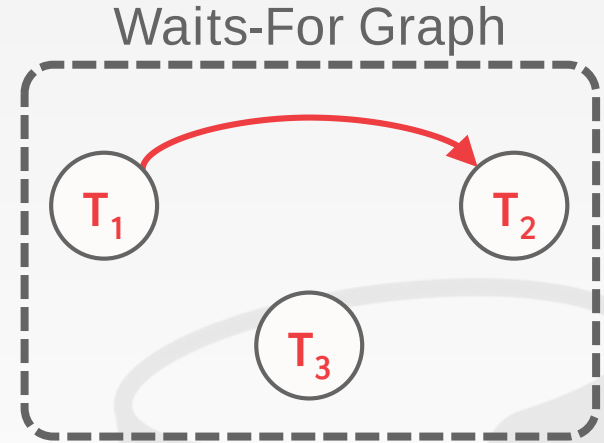
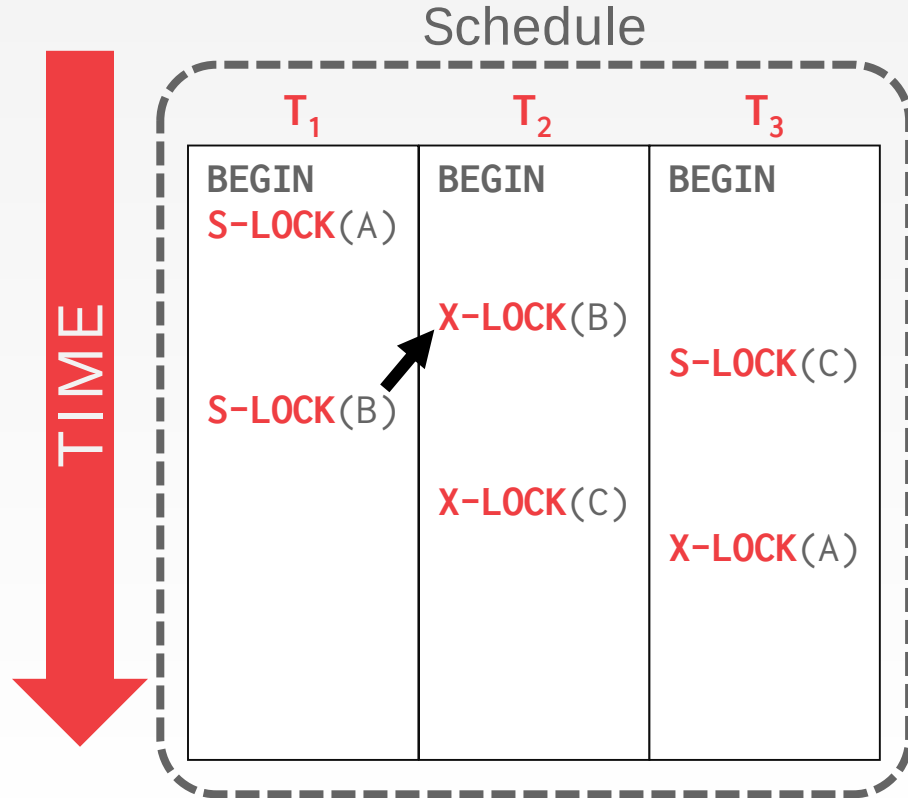
DEADLOCK DETECTION

The DBMS creates a **waits-for** graph to keep track of what locks each txn is waiting to acquire:

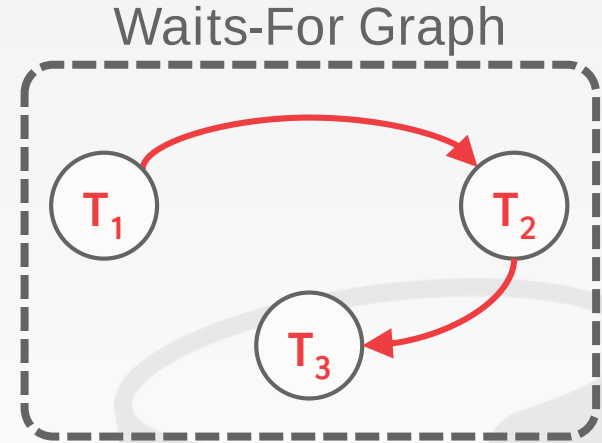
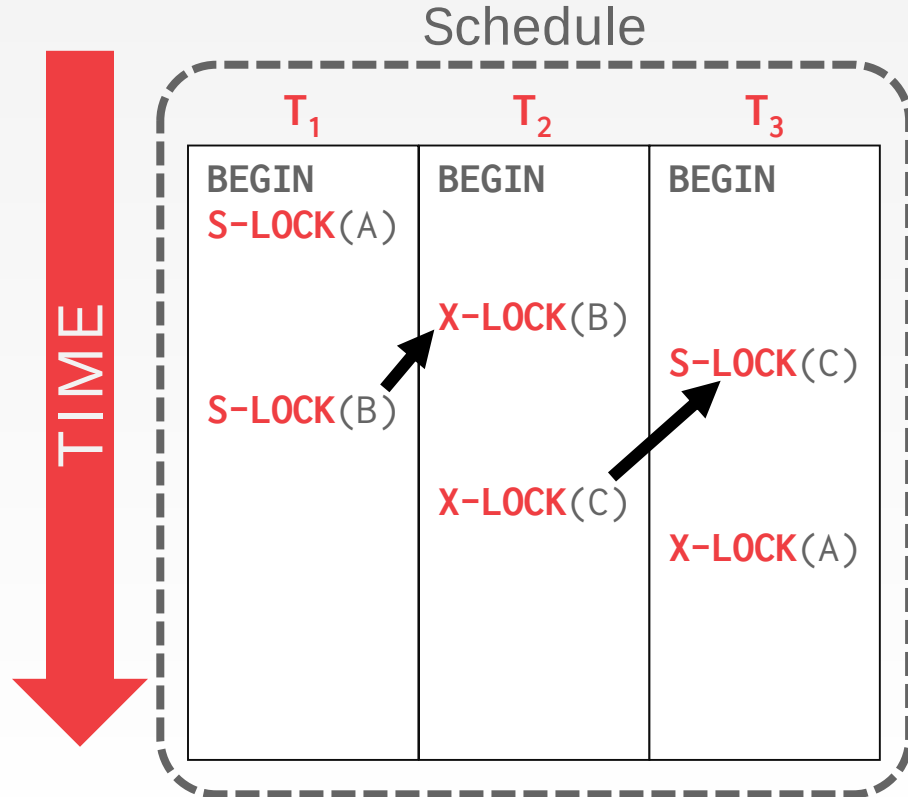
- Nodes are transactions
- Edge from T_i to T_j if T_i is waiting for T_j to release a lock.

The system periodically checks for cycles in ***waits-for*** graph and then decides how to break it.

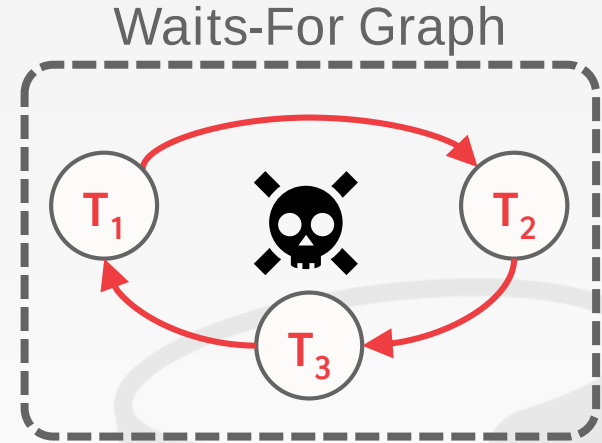
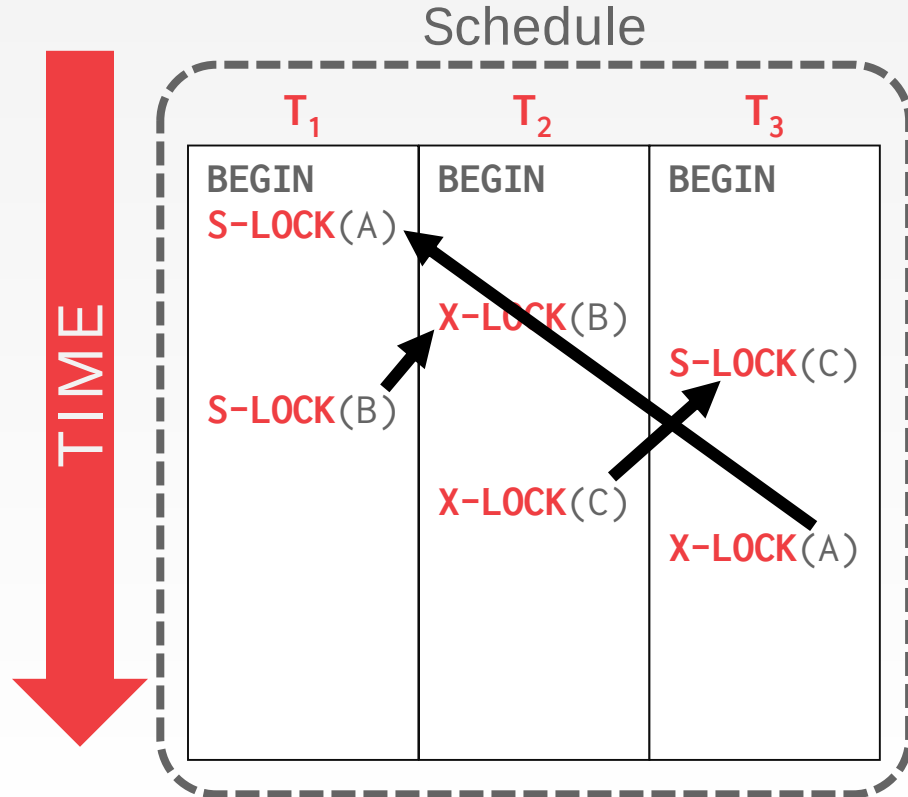
DEADLOCK DETECTION



DEADLOCK DETECTION



DEADLOCK DETECTION



DEADLOCK HANDLING

When the DBMS detects a deadlock, it will select a "victim" txn to rollback to break the cycle.

The victim txn will either restart or abort (more common) depending on how it was invoked.

There is a trade-off between the frequency of checking for deadlocks and how long txns have to wait before deadlocks are broken.

DEADLOCK HANDLING: VICTIM SELECTION

Selecting the proper victim depends on a lot of different variables....

- By age (lowest timestamp)
- By progress (least/most queries executed)
- By the # of items already locked
- By the # of txns that we have to rollback with it

We also should consider the # of times a txn has been restarted in the past to prevent starvation.

DEADLOCK HANDLING: ROLLBACK LENGTH

After selecting a victim txn to abort, the DBMS can also decide on how far to rollback the txn's changes.

Approach #1: Completely

Approach #2: Minimally



DEADLOCK PREVENTION

When a txn tries to acquire a lock that is held by another txn, the DBMS kills one of them to prevent a deadlock.

This approach does not require a *waits-for* graph or detection algorithm.

DEADLOCK PREVENTION

Assign priorities based on timestamps:

→ Older Timestamp = Higher Priority (e.g., $T_1 > T_2$)

Wait-Die ("Old Waits for Young")

→ If *requesting txn* has higher priority than *holding txn*, then *requesting txn* waits for *holding txn*.

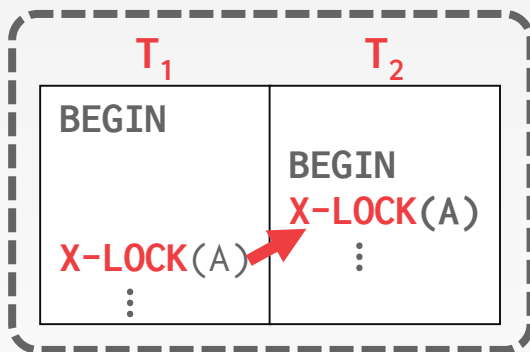
→ Otherwise *requesting txn* aborts.

Wound-Wait ("Young Waits for Old")

→ If *requesting txn* has higher priority than *holding txn*, then *holding txn* aborts and releases lock.

→ Otherwise *requesting txn* waits.

DEADLOCK PREVENTION

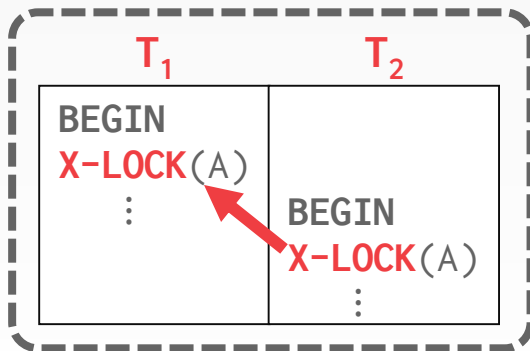


Wait-Die

T_1 waits

Wound-Wait

T_2 aborts



Wait-Die

T_2 aborts

Wound-Wait

T_2 waits

DEADLOCK PREVENTION

Why do these schemes guarantee no deadlocks?

Only one "type" of direction allowed when waiting for a lock.

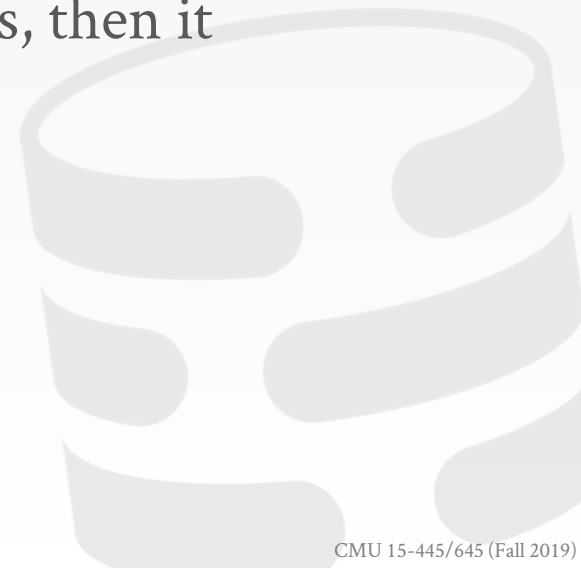
When a txn restarts, what is its (new) priority?

Its original timestamp. Why?

OBSERVATION

All of these examples have a one-to-one mapping from database objects to locks.

If a txn wants to update one billion tuples, then it has to acquire one billion locks.



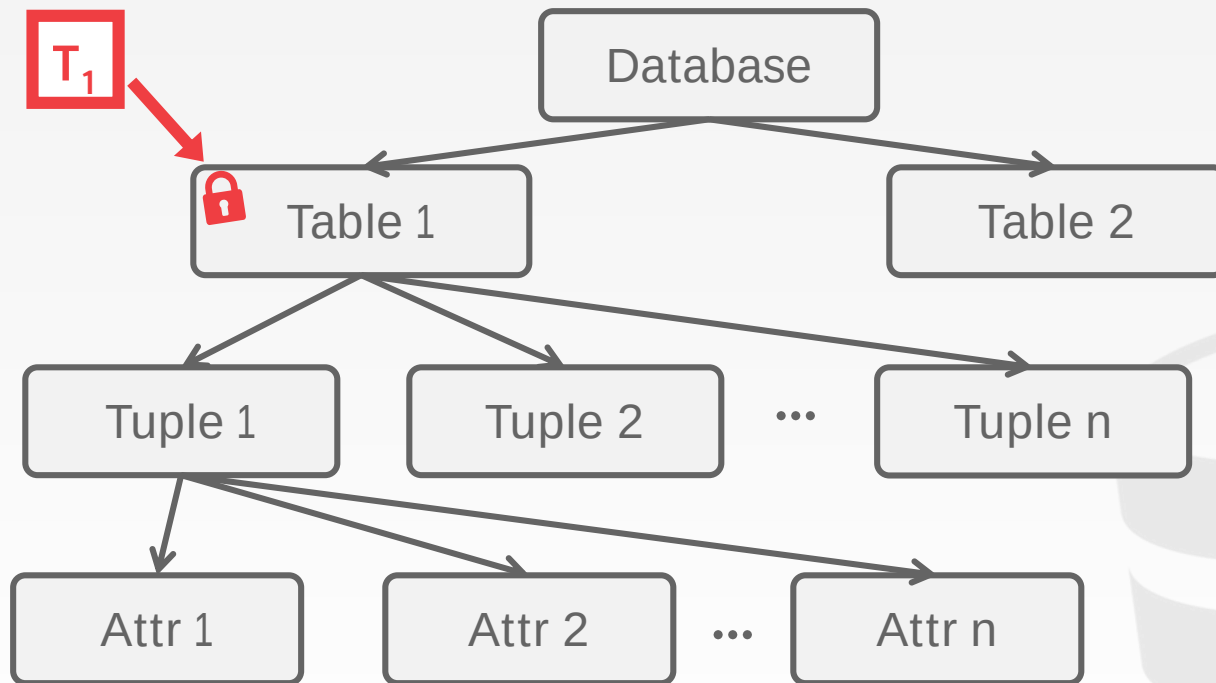
LOCK GRANULARITIES

When we say that a txn acquires a “lock”, what does that actually mean?

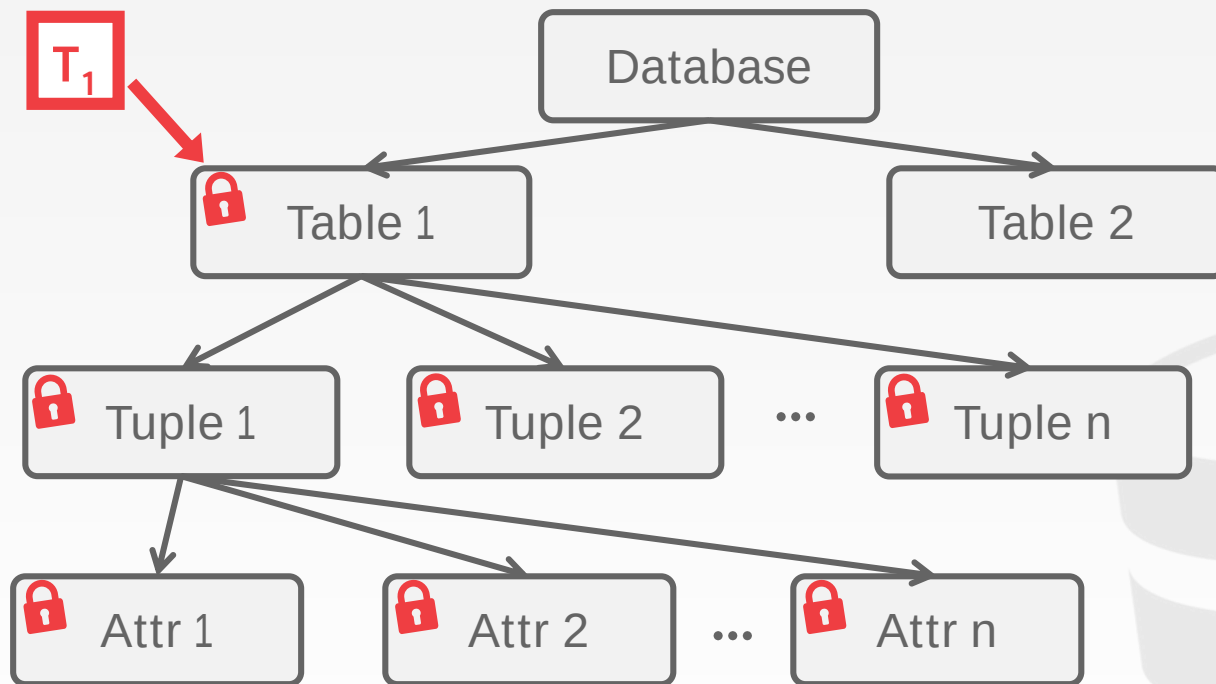
→ On an Attribute? Tuple? Page? Table?

Ideally, each txn should obtain fewest number of locks that is needed...

DATABASE LOCK HIERARCHY



DATABASE LOCK HIERARCHY



EXAMPLE

T_1 – Get the balance of Andy's shady off-shore bank account.

T_2 – Increase Matt's bank account balance by 1%.

What locks should these txns obtain?



EXAMPLE

T_1 – Get the balance of Andy's shady off-shore bank account.

T_2 – Increase Matt's bank account balance by 1%.

What locks should these txns obtain?

Multiple:

- Exclusive + Shared for leafs of lock tree.
- Special Intention locks for higher levels.

INTENTION LOCKS

An **intention lock** allows a higher level node to be locked in **shared** or **exclusive** mode without having to check all descendent nodes.

If a node is in an intention mode, then explicit locking is being done at a lower level in the tree.

INTENTION LOCKS

Intention-Shared (IS)

→ Indicates explicit locking at a lower level with shared locks.

Intention-Exclusive (IX)

→ Indicates locking at lower level with exclusive or shared locks.

INTENTION LOCKS

Shared+Intention-Exclusive (**SIX**)

- The subtree rooted by that node is locked explicitly in **shared** mode and explicit locking is being done at a lower level with **exclusive-mode** locks.

COMPATIBILITY MATRIX

		T_2 Wants				
T_1 Holds		IS	IX	S	SIX	X
	IS	✓	✓	✓	✓	×
	IX	✓	✓	×	×	×
	S	✓	×	✓	×	×
	SIX	✓	×	×	×	×
	X	×	×	×	×	×

LOCKING PROTOCOL

Each txn obtains appropriate lock at highest level of the database hierarchy.

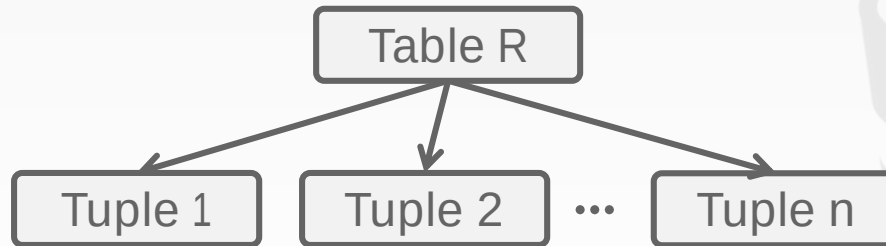
To get **S** or **IS** lock on a node, the txn must hold at least **IS** on parent node.

To get **X**, **IX**, or **SIX** on a node, must hold at least **IX** on parent node.

EXAMPLE – TWO-LEVEL HIERARCHY

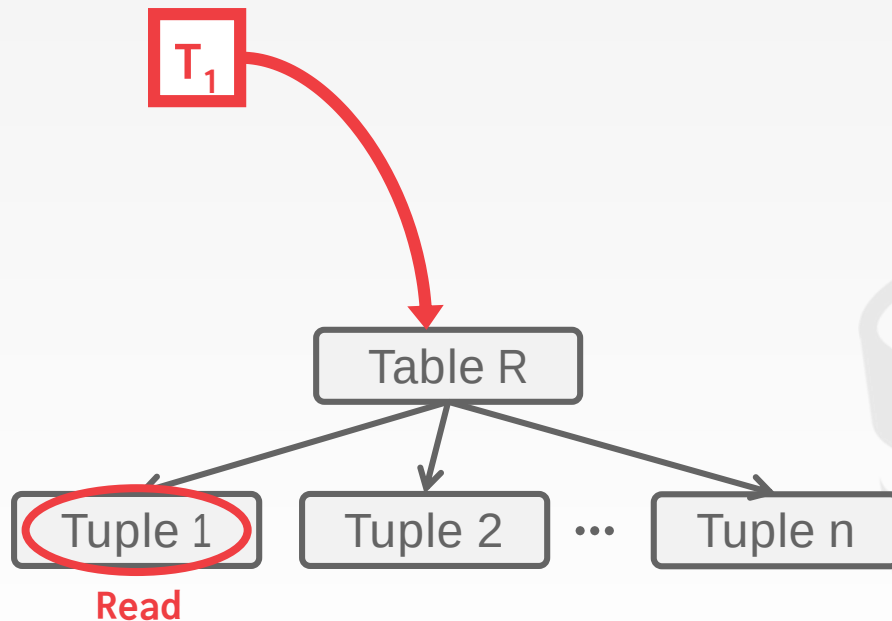
Read Andy's record in R.

T₁



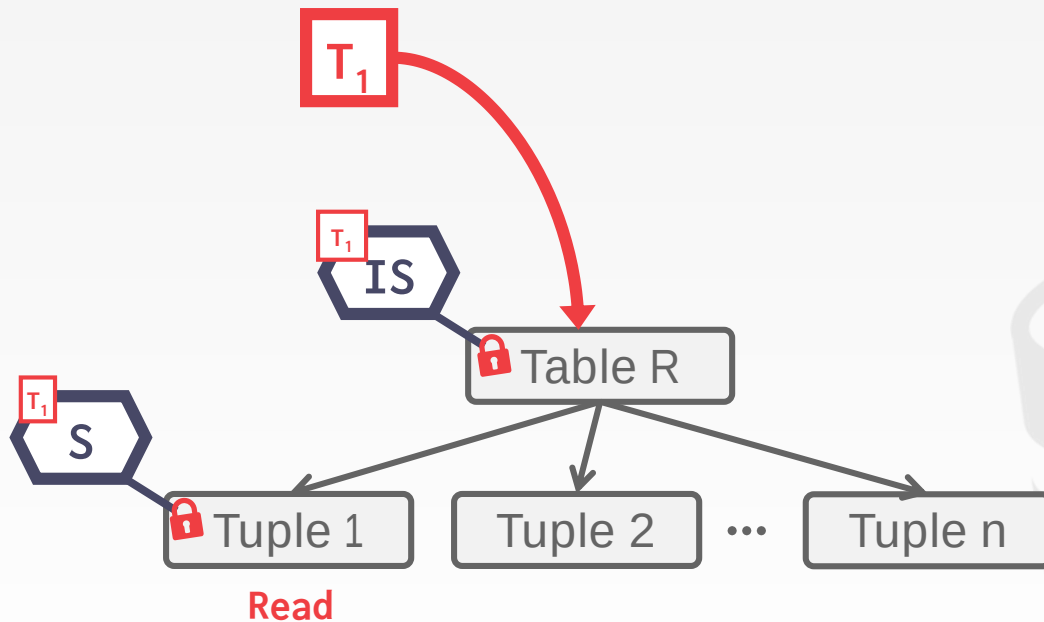
EXAMPLE – TWO-LEVEL HIERARCHY

Read Andy's record in R.



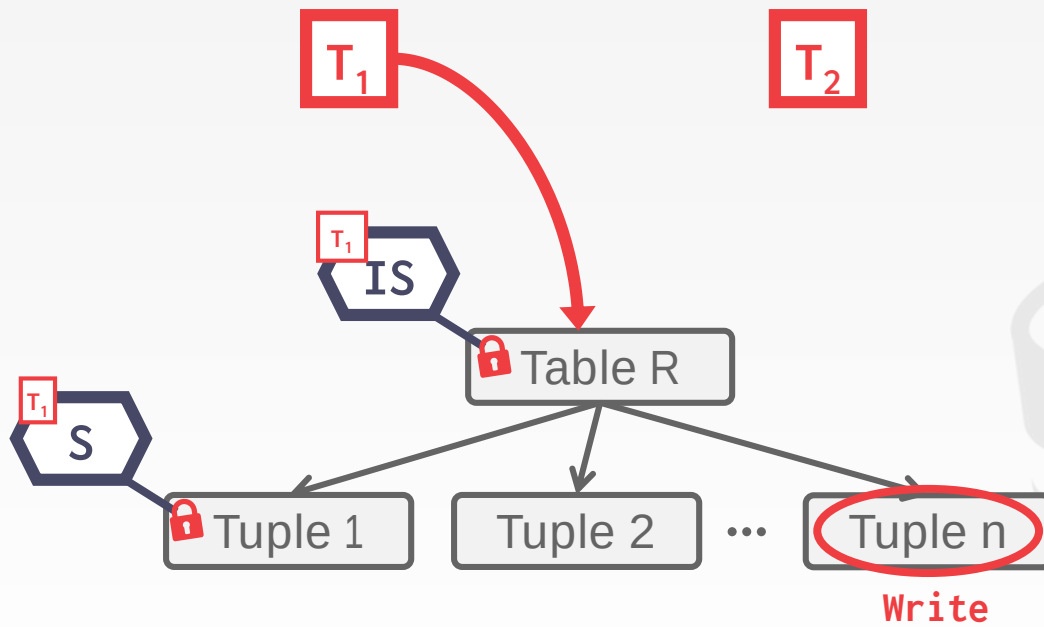
EXAMPLE – TWO-LEVEL HIERARCHY

Read Andy's record in R.



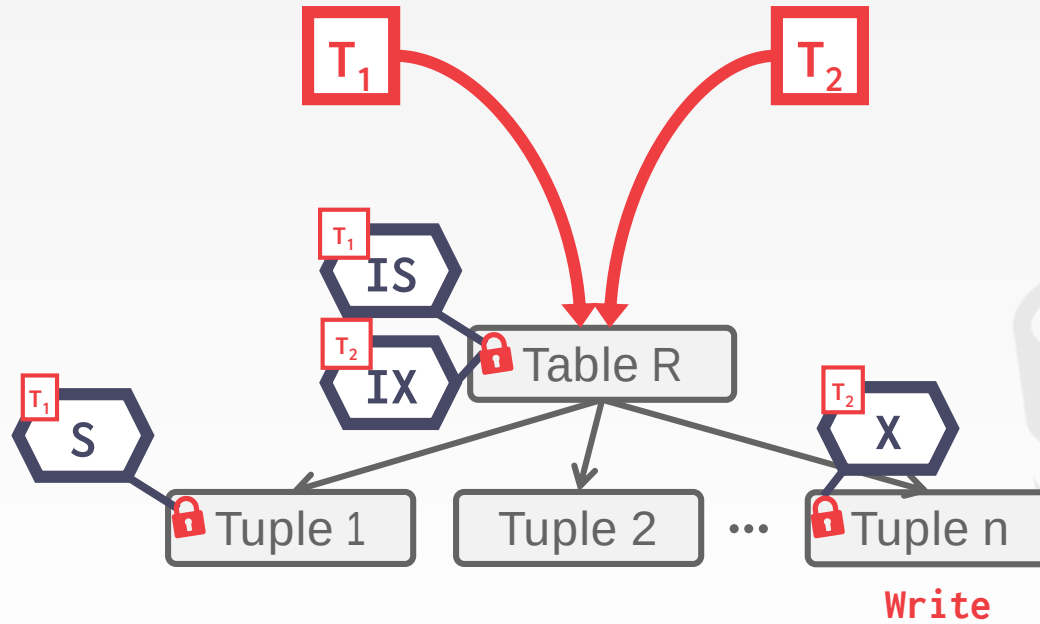
EXAMPLE – TWO-LEVEL HIERARCHY

Update Matt's record in R.



EXAMPLE – TWO-LEVEL HIERARCHY

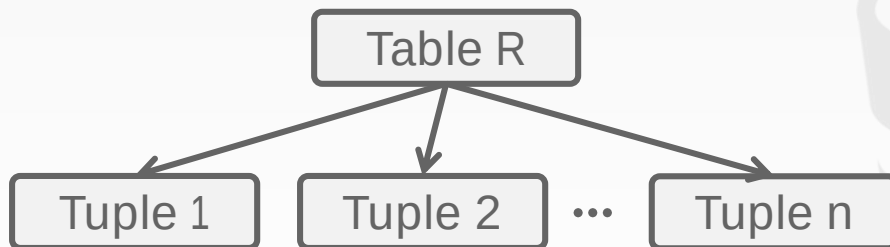
Update Matt's record in R.



EXAMPLE – THREESOME

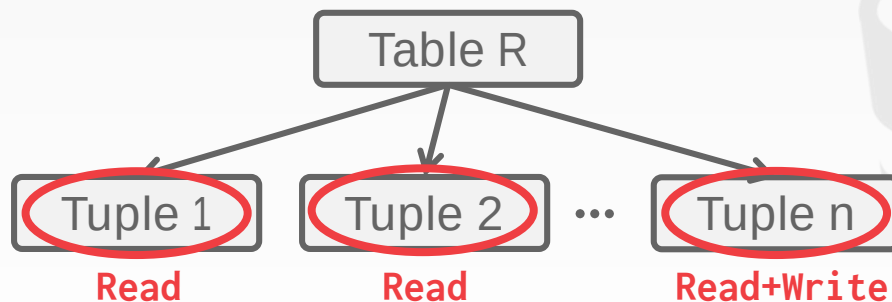
Assume three txns execute at same time:

- T_1 – Scan **R** and update a few tuples.
- T_2 – Read a single tuple in **R**.
- T_3 – Scan all tuples in **R**.



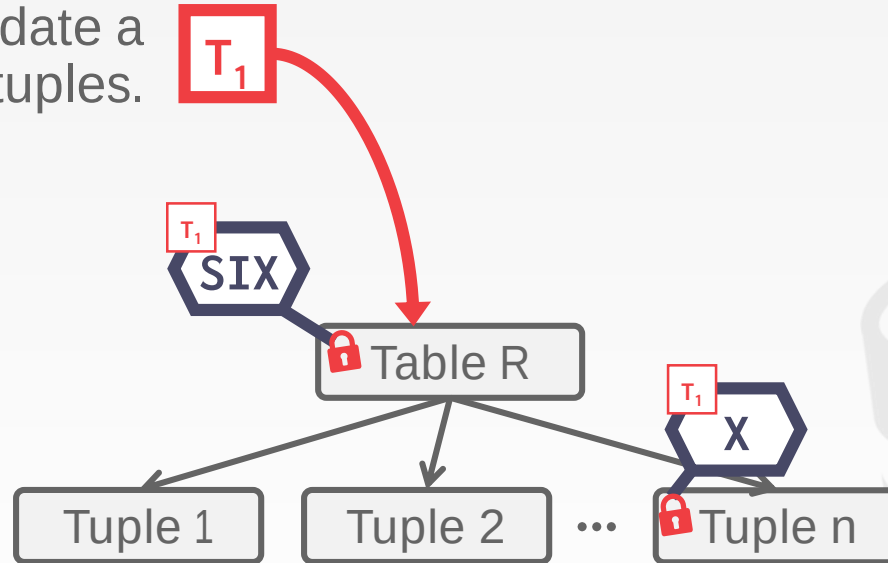
EXAMPLE – THREESOME

Scan **R** and update a few tuples. **T₁**

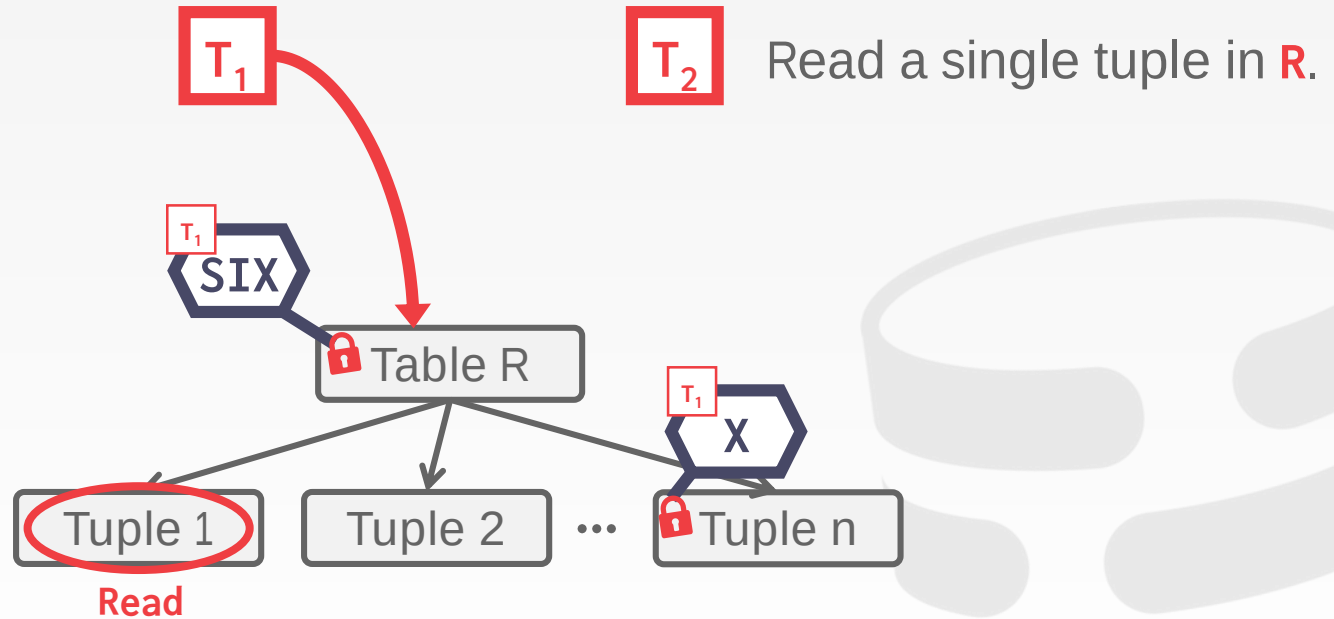


EXAMPLE – THREESOME

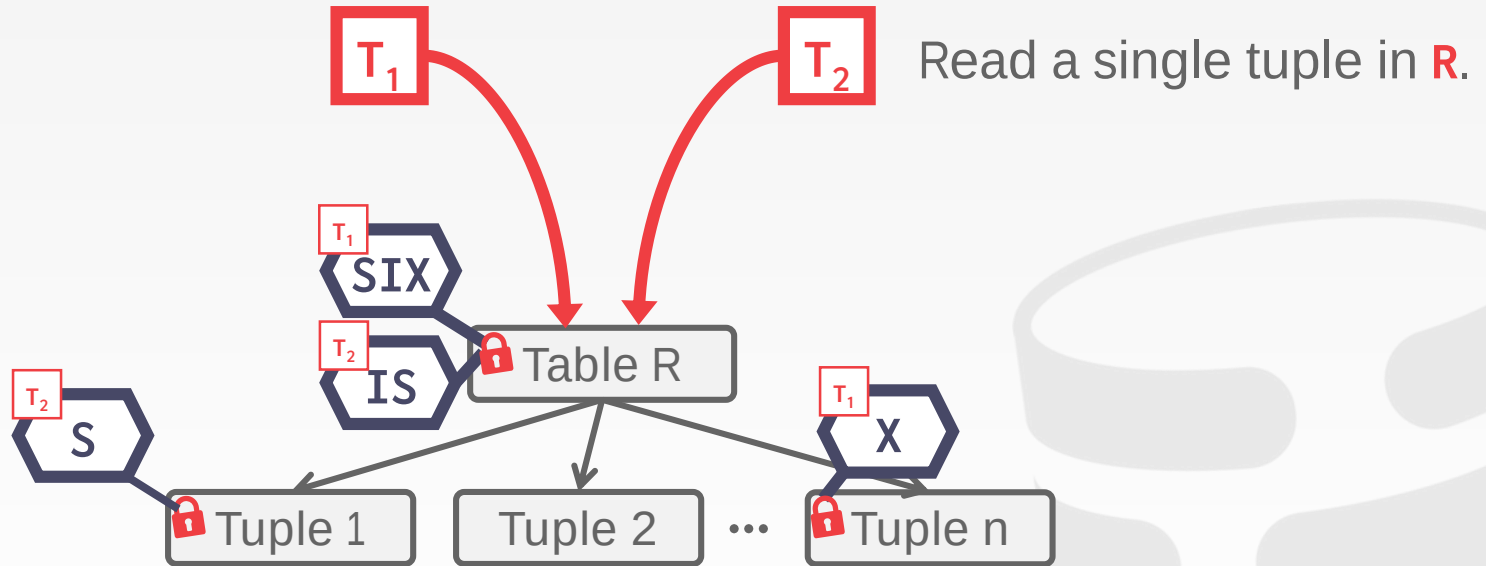
Scan **R** and update a few tuples.



EXAMPLE – THREESOME

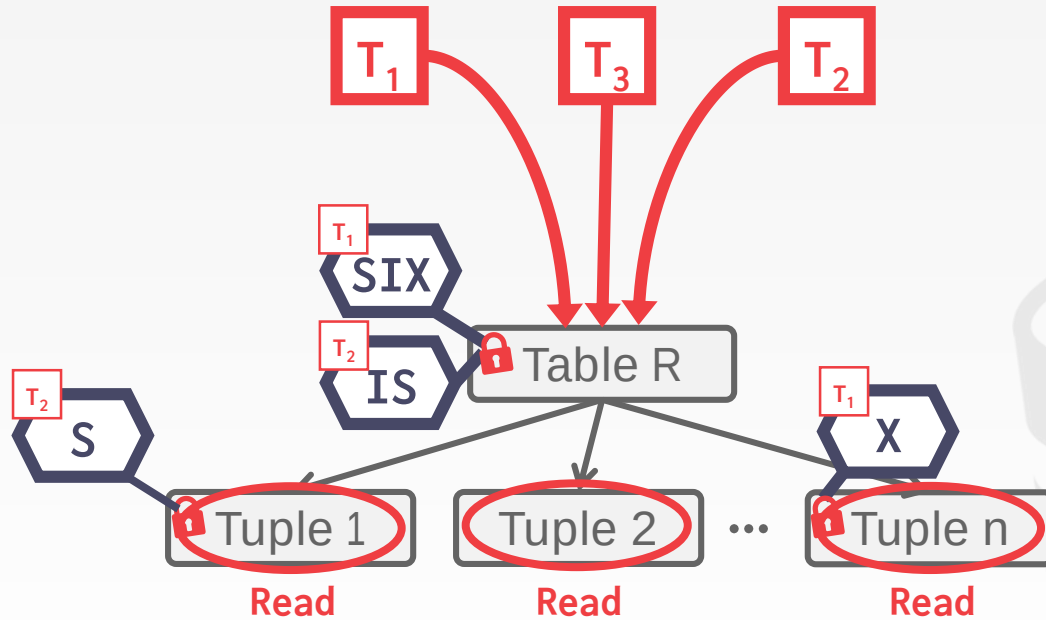


EXAMPLE – THREESOME



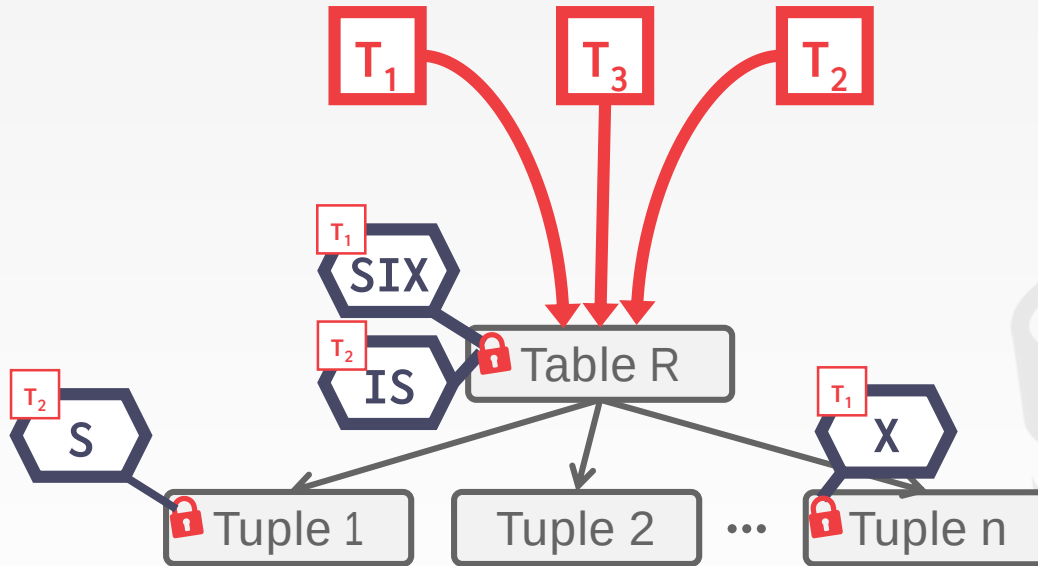
EXAMPLE – THREESOME

Scan all tuples in **R**.



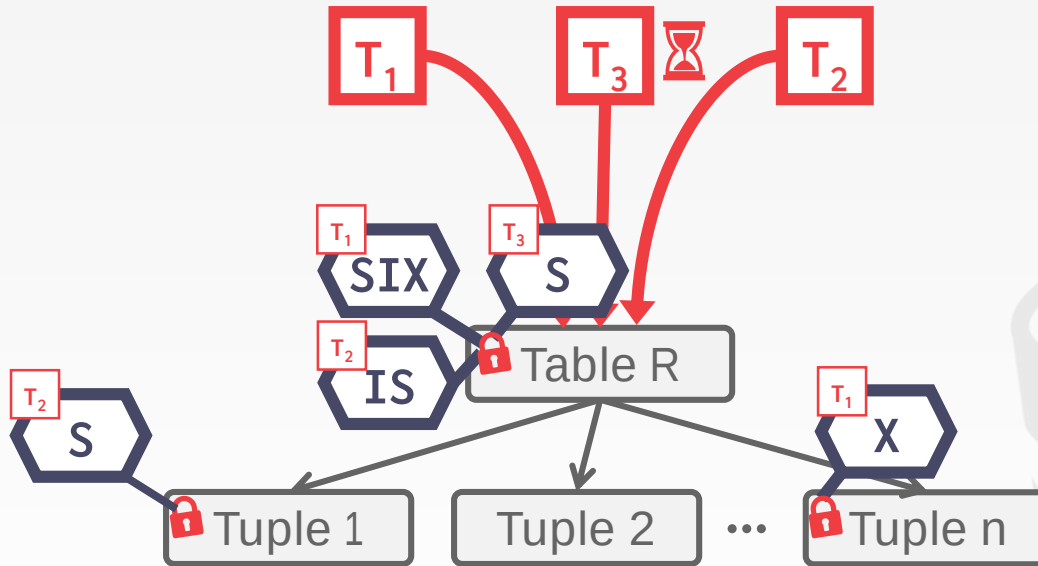
EXAMPLE – THREESOME

Scan all tuples in **R**.



EXAMPLE – THREESOME

Scan all tuples in **R**.



MULTIPLE LOCK GRANULARITIES

Hierarchical locks are useful in practice as each txn only needs a few locks.

Intention locks help improve concurrency:

- **Intention-Shared (IS)**: Intent to get **S** lock(s) at finer granularity.
- **Intention-Exclusive (IX)**: Intent to get **X** lock(s) at finer granularity.
- **Shared+Intention-Exclusive (SIX)**: Like **S** and **IX** at the same time.

LOCK ESCALATION

Lock escalation dynamically asks for coarser-grained locks when too many low level locks acquired.

This reduces the number of requests that the lock manager has to process.

LOCKING IN PRACTICE

You typically don't set locks manually in txns.

Sometimes you will need to provide the DBMS with hints to help it to improve concurrency.

Explicit locks are also useful when doing major changes to the database.

LOCK TABLE

Explicitly locks a table.

Not part of the SQL standard.

→ Postgres/DB2/Oracle Modes: **SHARE**, **EXCLUSIVE**

→ MySQL Modes: **READ**, **WRITE**



```
LOCK TABLE <table> IN <mode> MODE;
```

```
SELECT 1 FROM <table> WITH (TABLOCK, <mode>);
```

```
LOCK TABLE <table> <mode>;
```

SELECT...FOR UPDATE

Perform a select and then sets an exclusive lock on the matching tuples.

Can also set shared locks:

- Postgres: **FOR SHARE**
- MySQL: **LOCK IN SHARE MODE**

```
SELECT * FROM <table>  
WHERE <qualification> FOR UPDATE;
```

CONCLUSION

2PL is used in almost DBMS.

Automatically generates correct interleaving:

- Locks + protocol (2PL, SS2PL ...)
- Deadlock detection + handling
- Deadlock prevention

NEXT CLASS

Timestamp Ordering Concurrency Control



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]